Claude SDK for Pharo: Technical Specification

Punt Labs

April 2026 — v0.3.0-dev

Contents

I Learn about Claude **2**

1 Overview **2**

2 Design Goals **3**

3 Scope **4**

II API Reference **5**

4 Messages API **5**

4.1 Request Lifecycle 6

4.2 Sending a Message 6

4.3 Request Parameters (ClaudeMessageRequest) 7

4.4 Response Shape (ClaudeMessage) 9

4.5 Content Block Types 10

4.6 Per-Request Options 11

5 Messages Examples **12**

6 Count Tokens **13**

7 Models API — List Models **13**

8 Models API — Get a Model **14**

9 Handling Stop Reasons **15**

10 Errors **16**

11 API Versions 17

12 Beta Headers 18

13 Rate Limits 19

14 Service Tiers 20

15 Client SDKs 21

16 Batches API 21

17 Files API 21

18 Skills API 23

19 Prompt Tools API 25

20 Admin API 25

III Build with Claude 26

21 Streaming Messages 26

21.1 SSE Protocol 26

21.2 Event Types 27

21.3 SSE Decoder 27

21.4 Message Accumulator 29

22 Prompt Caching 30

23 Extended Thinking 31

24 Context Editing 33

25 Vision / Multimodal 33

26 PDF / Document Block 34

27 Citations 35

28 Search Results 37

29 Structured Outputs 37

30 Token Counting (Feature Guide) 40

31 Batch Processing 42

IV Agents and Tools 42

32 Tool Use 42

33 Server Tools 43

34 MCP Connector 45

35 Agent Skills 46

About this Specification

Section order mirrors Anthropic's Messages API docs (https://docs.claude.com/en/api/messages). Unimplemented surface is marked **Gap (Phase 2)**; see `claude-sdk-specification-gap-analysis.md`. Pharo-specific implementation notes live in *claude-sdk-specification-pharo-notes.tex*. The Agent layer (tool runner, sub-agents, dashboard) is in the separate *Claude Agent SDK for Pharo — Technical Specification*.

Part I

Learn about Claude

1 Overview

Anthropic's Messages API is the primary endpoint for chat-style generation against Claude models. A caller sends an ordered list of user and assistant turns; the API returns a typed response carrying assistant text, tool-use blocks, citations, stop reason, and token usage. Streaming is a transport variant of the same shape, delivering the response as Server-Sent Events (SSE). Synchronous chat-style generation, tool-use loops, and streaming interactions route through this endpoint; asynchronous batch processing reuses the Messages request shape but submits via the Batches API instead. The other endpoint families — Files, Skills, Admin, Batches, Prompt Tools, Models, and Count Tokens — live as separate surfaces that compose with Messages or stand alone.

The API is built for the workloads people actually run against Claude: agents that loop tool calls back into the model, copilots that assist a developer in an editor, chat UIs that hold a turn-based conversation, retrieval-augmented generation pipelines that ground answers in fetched documents, document Q&A, classification, summarization, structured extraction, and content generation. It is also the backbone for the rest of Anthropic's platform: server-side tools attach via the `tools` array in the request body (no beta), MCP connectors attach via both the `mcp_servers` request body field and the `mcp-client-2025-04-04` beta header, and Files and Skills attach via their own beta headers (`files-api-2025-04-14`, `skills-2025-10-02`) that opt the request into a feature group.

The Claude SDK for Pharo is a Smalltalk client for that API, running inside a live Pharo 12 image. It gives a Smalltalker the same surface a Python or TypeScript developer gets from the first-party SDKs — build a message, send it, read Claude's response as a typed object — while keeping to idiomatic Smalltalk conventions throughout: fluid class syntax, typed hierarchies instead of dictionaries, `>>>` doctest examples, Iceberg-managed Tonel packages. The SDK ships in two layers. This document specifies the *Messaging layer*: `ClaudeClient`, the `ClaudeMessage` / `ClaudeContentBlock` hierarchy, the error classes, the streaming decoder, the beta-gated APIs (Files, Skills, MCP). The *Agent layer* — tool runner, sub-agents, dashboard, rate limiter — builds on this surface and is described in the separate *Claude Agent SDK for Pharo* specification.

The SDK covers the public Messages API endpoints at `https://api.anthropic.com` (configurable via `baseUrl` on `ClaudeClient` for proxies or mocks): `POST /v1/messages` for creating messages (streaming and non-streaming), `POST /v1/messages/count_tokens` for token counting, `GET /v1/models` and `/v1/models/{id}` for model discovery, and the Files API endpoints (`/v1/files` for upload, list, get, download, and delete). The endpoint map in full:

Endpoint	Method	Description
/v1/messages	POST	Create message (streaming and non-streaming)
/v1/messages/count_tokens	POST	Count tokens for a message
/v1/models	GET	List available models
/v1/models/{id}	GET	Get model details
/v1/files	POST	Upload a file
/v1/files	GET	List uploaded files
/v1/files/{id}	GET	Get file metadata
/v1/files/{id}/content	GET	Download file content
/v1/files/{id}	DELETE	Delete a file

The example below is the canonical first SDK call: build a client with an API key, build a request with a model and a user message, send it, and read the text off the response. Every later section elaborates on one piece of this shape — request options, streaming, tool use, prompt caching, thinking — but a reader who gets this example running has the whole SDK in hand. The example evaluates to Claude's reply text as a String:

```
| client request response |
"Build a client (see ClaudeSDKExampleSupport resolveClient for keyring
 resolution in scripts; the apiKey: factory is the minimal shape)."
client := ClaudeClient apiKey: 'sk-ant-...'.

"Build a request: model, max tokens, one user turn."
request := ClaudeMessageRequest new
    model: ClaudeModel opus46;
    maxTokens: 1024;
    addUserMessage: 'Write a haiku about Smalltalk.';
    yourself.

"Synchronous POST to /v1/messages; returns a ClaudeMessage."
response := client sendMessage: request.

"textContent joins all text blocks into a single String."
response textContent
```

Pharo-specific implementation notes — threading discipline around the Morphic UI, Iceberg / Tonel packaging, image save and restore behavior, supervised process patterns — live in the companion document *Claude SDK for Pharo: Pharo-Specific Implementation Notes*.

For the full shape of the Messages API surface see §4; for streaming see §21; for thinking see §23.

Status: *Available*. Anthropic docs: https://docs.claude.com/en/api/messages.

2 Design Goals

The SDK is shaped by four commitments that trade off against each other in predictable ways: parity with the Messages API defines the surface, idiomatic Smalltalk defines the style, Pharo-safe threading defines the concurrency contract, and a minimal dependency footprint keeps the package graph light enough to load into any Pharo 12 image. The numbered list below names each commitment and the specific choices that follow from it.

G1. Feature parity with Anthropic's Messages API. Every v1.0 Pharo-scope feature in §3 maps to a typed Smalltalk surface: the Messages API itself (send, stream, tool use), the streaming decoder, models API, token counting, the error hierarchy, core options, extended thinking, citations, server tool results, server-side tools, the MCP connector, Skills API, Files API, output encoding, and the message-decomposition primitives (turn, stop details, request context, container). Both stable and

beta APIs ship together — beta headers are managed per §12, automatic for Files and Skills, opt-in via `#betas:` for others — so a caller can reach the full v1.0 surface without conditional code paths.

G2. Idiomatic Smalltalk. Follow Kent Beck's *Smalltalk Best Practice Patterns*: composed methods, intention-revealing selectors, pluggable behavior, collecting parameters. Use the class hierarchy to carry type information rather than dictionaries or type tags — `ClaudeMessage` and `ClaudeContentBlock` are families with polymorphic dispatch, not schema-bag records. JSON serialization is explicit (`asJson` / `fromJson:` with a `jsonKeyMap`) rather than reflective. The SDK should read like Smalltalk to a Smalltalker.

G3. Pharo-safe threading. Network I/O must never block the Morphic UI thread. The SDK itself is synchronous, so every caller knows exactly which selectors block and for how long; all blocking operations are documented so consumers know what to fork with `forkAt:` and what to defer back to the UI via `WorldState defer:`. The companion Pharo-notes document covers the forking patterns.

G4. Minimal dependencies. Use only Pharo's built-in libraries: Zinc for HTTP, STONJSON for JSON, and the standard collections. No external Metacello baselines, no FFI to OS libraries from the Messaging layer. The SDK loads into a stock Pharo 12 image by filing in the `Claude-Messaging-*` Tonel packages from `src/` (driven by `make setup` / `make filein`, which use `TonelReader` and `MCPackageLoader`); per-product Metacello baselines are future work.

This section is reference material; no runnable example applies.

Status: *Available*.

3 Scope

This SDK covers the public Messages API plus the beta APIs that stabilized as of v1.0. The table below enumerates each feature with its v1.0 Pharo scope and the version the Pharo surface landed in. Features marked for a later version column ship later — the SDK reserves version slots so callers can track when a surface becomes available rather than guessing from a changelog. Features marked *Out of scope* belong to adjacent Anthropic surfaces (Bedrock, Vertex, legacy completions) or to the Agent layer; the scope boundary is deliberate and documented per row.

Feature	Smalltalk Scope	Version
Messages API	Full (send, stream, tool use)	v1
Streaming (SSE)	Full (decoder, events, accumulator)	v1
Models API	List, get, model info type	v1
Token counting	CountTokens method and return type	v1
Error handling	Full hierarchy, retry, rate limits	v1
Options system	Core options (key, URL, timeout)	v1
Extended thinking	ThinkingConfig, thinking blocks, redacted	v1
Citations	Citation types on TextBlock	v1
Server tool results	Read-only response types	v1
Server-side tools	Full (web search, web fetch, code exec, bash, text editor, memory, tool search)	v1
MCP connector	Full (request param + response content blocks)	v1
Skills API	Full (CRUD + versions + container loading + multipart upload)	v1
Files API	Full (upload, list, get, download, delete)	v1
Output encoding	Text output format (encoding + markup)	v1
Message decomposition	Turn, stop details, request context, container	v1
Tool runner	See *Claude Agent SDK Specification*	v1
Message batches	Not yet implemented	v2
Bedrock / Vertex	Out of scope	—
Completions (legacy)	Out of scope	—

The Batches API row is tracked against the full specification in §16; the Files API lands in §17.

This section is reference material; no runnable example applies.

Status: *Available*. Anthropic docs: https://docs.claude.com/en/api/messages.

Part II

API Reference

4 Messages API

The Messages API is Claude's primary generation endpoint — every synchronous chat-style interaction, tool-use loop, and streaming response rides on `POST /v1/messages`. The Pharo SDK exposes that endpoint through three collaborating objects: `ClaudeClient` holds authentication and connection configuration, `ClaudeMessageRequest` builds the request body (model, messages, options), and `ClaudeMessage` comes back as the typed response. Any synchronous chat-style generation, tool-use loop, or streaming interaction routes through here; asynchronous batch processing reuses the same Messages request shape but submits via the Batches API endpoints instead. The other endpoint families (Files, Skills, Admin, Batches, Prompt Tools, Models, Count Tokens) live as separate HTTP endpoint surfaces that compose with Messages or stand alone. Thinking, tool use, streaming, prompt caching, and citations are all features layered on top of a Messages request.

A `ClaudeClient` is cheap to build and meant to be reused. The simplest form takes an API key; the three-keyword factory adds a base URL override (for proxies or mock servers) and a timeout. `fromEnvironment` reads the `ANTHROPIC_API_KEY` variable via `Smalltalk os environment` and raises if it is unset — useful for scripts and CI runners where the key is injected from outside the image.

```
"Explicit API key."
client := ClaudeClient apiKey: 'sk-ant-...'.

"From environment variable ANTHROPIC_API_KEY."
client := ClaudeClient fromEnvironment.

"With a proxy or mock server URL plus a tighter timeout."
client := ClaudeClient
    apiKey: 'sk-ant-...'
    baseUrl: 'https://custom-proxy.example.com'
    timeout: 120.
```

The client's configuration lives in eight instance variables, all with defaults set by `initialize`:

Variable	Type	Default
`apiKey`	String	From `#apiKey:` argument or `fromEnvironment`
`baseUrl`	String	`https://api.anthropic.com`
`apiVersion`	String	`2023-06-01`
`timeout`	Integer	120 (seconds); used for non-streaming requests
`streamingTimeout`	Integer	300 (seconds); used for `#streamMessage:do:` and as the socket read timeout in `#openStreamingSocket:`
`maxRetries`	Integer	5
`rateLimiter`	`ClaudeRateLimiter` or nil	Optional shared rate limiter; populated when the client joins an Agent-layer rate limiter
`debugLogBlock`	Block or nil	Optional callable that receives a Dictionary per request and response event (keys `#direction`, `#url`, `#body`, `#status`). Wire with `#debugLog:`.

A fresh `ZnClient` is created for each request rather than pooled, so every call performs its own TCP/TLS handshake. That is acceptable for an API whose typical conversation turn makes one to five requests; it also means `ClaudeClient` survives Pharo image save without special handling — no connections are held across save. A process that was mid-request at save time wakes up on a broken socket after restore; long-running consumers should guard their network forks accordingly.

4.1 Request Lifecycle

Every Messages request follows the same seven-step pipeline, and understanding it is how you reason about retry, timeout, and error shape. The request object is serialized to JSON with `STONJSON toString:`; a fresh `ZnClient` is configured with `x-api-key`, `anthropic-version: 2023-06-01`, `content-type: application/json`, and `accept: application/json`; the POST fires; any non-2xx response is inspected for retry eligibility and either retried with backoff (see §10) or raised as a typed `ClaudeApiError` subclass; the response body is parsed with `STONJSON fromString:`; the resulting dictionary is handed to `ClaudeMessage class>>fromJson:`; and the typed `ClaudeMessage` comes back to the caller. The HTTP POST and response-parse steps are blocking network I/O. The SDK itself never forks — consumers choose the concurrency strategy that fits their application, which means a call from the Morphic UI thread must be wrapped in `forkAt:` to avoid locking the UI for the duration of the request.

4.2 Sending a Message

The end-to-end example below covers the `sendMessage:` path against a `ClaudeMockServer` so it runs without an API key or network. Real code swaps the mock for `ClaudeSDKExampleSupport resolveClient` and removes the `baseUrl:` override. The request exercises three delegated setters beyond §1's minimal hello call: a system prompt, a sampling temperature, and a metadata tag that the server echoes in usage analytics. The example evaluates to an array with three fields pulled off the typed response — text, stop reason symbol, and input-token count:

```
| server client request response |
"Spin up an in-image mock HTTP server so the example needs neither network nor API key."
server := ClaudeMockServer startOn: 0.
[ "Register the canned response for POST /v1/messages."
  server
    respondTo: '/v1/messages'
    with: '{"id":"msg_01","type":"message","role":"assistant","content":[{"type":"text","text
        ":"2 plus 2 is 4."}],"model":"claude-sonnet-4-5","stop_reason":"end_turn","stop_sequence
        ":null,"usage":{"input_tokens":18,"output_tokens":7,"cache_creation_input_tokens":0,"
        cache_read_input_tokens":0}}'.
```

```
  "Build a ClaudeClient that talks to the mock."
  client := ClaudeClient
    apiKey: 'test-key'
    baseUrl: 'http://localhost:' , server port printString
    timeout: 5.
  "Build the request with a system prompt, a low temperature, and a metadata tag."
  request := ClaudeMessageRequest new
    model: ClaudeModel sonnet45;
    maxTokens: 256;
    system: 'You are a concise arithmetic tutor.';
    temperature: 0.0;
    metadata: (Dictionary newFromPairs: { 'user_id'. 'example-reader' });
    addUserMessage: 'What is 2 + 2?';
    yourself.
  "Synchronous POST to /v1/messages; returns a ClaudeMessage."
  response := client sendMessage: request.
  ^ { response textContent. response stopReason. response usage inputTokens } ]
    ensure: [ server stop ] "Always stop the mock, even on error."
"=> #('2 plus 2 is 4.' #end_turn 18)"
```

Application code wraps the `sendMessage:` call in an `on: ClaudeApiError do:` handler when it wants to react to specific error subclasses; for most callers the default behavior (raise the typed exception) is correct because retryable errors are already retried inside the client (see §10).

4.3 Request Parameters (ClaudeMessageRequest)

`ClaudeMessageRequest` is the request body as a first-class Smalltalk object. It holds the two required fields directly (`model`, `maxTokens`) along with the conversation turns and system prompt, and delegates optional parameters to five composite value objects. Breaking the surface into composites (sampling knobs, tool parameters, thinking params, request options, output config) keeps the top-level class small enough to scan while letting each family of options evolve on its own schedule:

Variable	Type	Description
`model`	String	Model ID (required)
`maxTokens`	Integer	Max output tokens (required)
`messages`	OrderedCollection	Conversation turns
`system`	String or nil	System prompt
`sampling`	`ClaudeSamplingParams` or nil	Sampling knobs (temperature, topP, topK, stopSequences)
`toolParams`	`ClaudeToolParams` or nil	Tool declarations and tool-choice policy
`thinkingParams`	`ClaudeThinkingParams` or nil	Extended thinking mode (enabled, adaptive, disabled)
`requestOptions`	`ClaudeRequestOptions` or nil	Per-request controls (metadata, cache control, service tier, betas, MCP servers, container, etc.)
`outputConfig`	`ClaudeOutputConfig` or nil	Effort hint and output format (text or JSON schema)

The composites are initialized in `initialize`, so a bare `ClaudeMessageRequest new` already has empty sampling, tool, thinking, request-options, and output-config objects ready to accept setters. Callers never need to instantiate the composites themselves; the request forwards the common setters directly, and each forwarded message sets the field on the right composite:

Selectors	Held by	Wire field(s)
#temperature: #topP: #topK: #stopSequences:	ClaudeSamplingParams	temperature, top_p, top_k, stop_sequences
#tools: #toolChoice:	ClaudeToolParams	tools, tool_choice
#thinking:	ClaudeThinkingParams	thinking
#metadata: #cacheControl: #serviceTier: #inferenceGeo: #speed: #betas: #mcpServers: #container: #stream:	ClaudeRequestOptions	metadata, cache_control, service_tier, inference_geo, speed, anthropic-beta header, mcp_servers, container, stream
#outputFormat: #effort:	ClaudeOutputConfig	output_config.format, output_config.effort

Two fields do not live on the request even though new readers often expect them there. The `strict` flag is per-tool and lives on `ClaudeTool>>strict:` (or on any `ClaudeServerTool` subclass), because strictness is a property of the tool schema the model is asked to honor, not of the request that carries it. A server-assigned `requestId` is exposed response-side on `ClaudeRequestContext>>requestId` and reached via `message requestId`; the server fills it in, so it cannot exist on a request that has not yet been sent.

Output encoding is a Pharo-specific convenience built on top of the system prompt rather than a wire field. `ClaudeTextOutputFormat` combines two axes — `encoding` (`#ascii`, `#latin1`, `#utf8`) and `markup` (`#plain`, `#microdown`, `#markdown`, `#json`, `#html`, `#latex`) — into a prompt suffix that asks the model to produce text in that shape. The wire request carries no `output_format` field for text output; only `json_schema` has a dedicated wire surface. Configure the format on the request and the SDK appends the suffix during `asJson`:

```
"Ask for UTF-8-encoded Markdown output."
request outputFormat: (ClaudeTextOutputFormat new
    encoding: #utf8;
    markup: #markdown;
    yourself).
```

Tool choice is a Dictionary that tells the model whether it may use tools, must use some tool, or must use one specific named tool. The `auto` shape lets the model decide; the `any` shape forces a tool call without pinning which; the `tool` shape names the tool the model must call:

```
"Let the model decide (default)."
request toolChoice: (Dictionary new at: 'type' put: 'auto'; yourself).

"Force tool use without pinning which tool."
request toolChoice: (Dictionary new at: 'type' put: 'any'; yourself).

"Force one named tool."
request toolChoice: (Dictionary new
    at: 'type' put: 'tool';
    at: 'name' put: 'get_weather';
    yourself).
```

`ClaudeThinkingParams` wraps the extended-thinking configuration. Four class-side factories cover the modes: `#enabled:` turns thinking on with a token budget; `#adaptive` lets the model decide; `#adaptiveWithDisplay:` adds a display mode (`'summarized'` or `'full'`); `#disabled` turns it off. Pass the resulting Dictionary to `#thinking:` on the request:

```
"Always think, with a 10000-token budget."
request thinking: (ClaudeThinkingParams enabled: 10000) asDictionary.

"Let the model decide whether to think."
request thinking: ClaudeThinkingParams adaptive asDictionary.

"Adaptive with a display-mode hint."
```

```
request thinking: (ClaudeThinkingParams adaptiveWithDisplay: 'summarized') asDictionary.

"Thinking off."
request thinking: ClaudeThinkingParams disabled asDictionary.
```

When thinking is enabled or adaptive, responses may contain `ClaudeThinkingBlock` and `ClaudeRedactedThinkingBlock` content blocks. Redacted blocks appear when the API omits thinking content for safety reasons; they carry opaque `data` that callers must preserve when round-tripping conversations so the next request retains the server's signed prefix. The full thinking surface is described in §23.

Conversation turns are built through five builder methods. `#addUserMessage:` appends a plain-text user turn; `#addAssistantMessage:` appends a plain-text assistant turn (useful for conversation replay); `#addAssistantResponse:` appends a whole prior `ClaudeMessage`, preserving its content-block shape (thinking, citations, tool use all survive); `#addToolResult:content:` and `#addToolResult:content:isError:` append a user turn containing a tool result paired to the matching tool-use id.

4.4 Response Shape (ClaudeMessage)

`ClaudeMessage` is the typed response every successful Messages call returns. It holds seven instance variables — `id`, `type`, `turn`, `usage`, `stopDetails`, `executionContainer`, and `requestContext` — and exposes the familiar API shape through forwarding accessors that delegate to the composite value objects. The public-accessor surface:

Accessor	Returns	Description
`#id`	String	Unique message identifier (instVar)
`#type`	String	Always `'message'` (instVar)
`#role`	Symbol	`#assistant` for responses; delegated to `turn`
`#content`	OrderedCollection of `ClaudeContentBlock`	Text, tool use, thinking blocks; delegated to `turn`
`#model`	String	Model ID used; delegated to `turn`
`#stopReason`	Symbol	See stop reason values below; delegated to `stopDetails`
`#stopSequence`	String or nil	Matched stop sequence; delegated to `stopDetails`
`#usage`	`ClaudeUsage`	Token counts (instVar)

The `stopReason` value is a Symbol the SDK normalizes from the wire string: `#end_turn`, `#tool_use`, `#max_tokens`, `#stop_sequence`, `#pause_turn`, and `#refusal`. See §9 for per-value handling patterns. Alongside the accessors, four key methods cover the most-used response operations: `#asParam` converts the message back to request-param shape for multi-turn replay; `#accumulate:` builds the message incrementally from streaming events (see §21); `#textContent` concatenates every `ClaudeTextBlock` in `content` into a single String; and `#toolUseBlocks` selects the `ClaudeToolUseBlock` entries for tool-dispatch code to iterate.

The four composite value objects keep the response class narrow. `ClaudeTurn` groups the role-and-content pair, `ClaudeStopDetails` holds the stop-reason breakdown, `ClaudeRequestContext` carries request-level metadata, and `ClaudeExecutionContainer` tracks the code-execution container state when a request used skills or code execution:

Class	Responsibility	Key Fields
`ClaudeTurn`	Role and content grouping	`role`, `content`, `model`
`ClaudeStopDetails`	Stop reason breakdown	`stopReason`, `stopSequence`, `category`, `explanation`
`ClaudeRequestContext`	Request-level metadata	`requestId`, `outputFormat`
`ClaudeExecutionContainer`	Container execution context	`containerId`, `expiresAt`, `skills`

ClaudeUsage is the fifth composite and tracks the token counts every response carries — it is what cost-analysis and cache-effectiveness code reads after a call:

Variable	Type	Description
inputTokens	Integer	Tokens in request
outputTokens	Integer	Tokens in response
cacheCreationInputTokens	Integer	Prompt cache creation
cacheReadInputTokens	Integer	Prompt cache hits
inferenceGeo	String or nil	Inference region hint echoed by the server

4.5 Content Block Types

A response's content is a collection of ClaudeContentBlock instances, not a single string — Claude may return interleaved text, tool use, thinking, and server-tool-result blocks in one message. Each block is a subclass with its own instance variables and its own #asJson / #fromJson: pair. ClaudeContentBlock class>>fromJson: dispatches on the discriminator type field via a registry Dictionary mapping type strings to subclasses; unknown types fall back to a raw-JSON instance of the abstract class so forward-compatible payloads never lose data. Feature sections cross-reference this taxonomy: Vision (§25), PDF (§26), Citations (§27), Search Results (§28), Thinking (§23), and Tool Use (§32). The primary hierarchy — blocks callers can both send and receive — is:

Class	Wire Type	Key Fields
ClaudeContentBlock	(abstract)	type, rawJson
ClaudeTextBlock	text	text, citations, cacheControl
ClaudeToolUseBlock	tool_use	id, name, input
ClaudeToolResultBlock	tool_result	toolUseId, content, isError, cacheControl
ClaudeThinkingBlock	thinking	thinking, signature
ClaudeRedactedThinkingBlock	redacted_thinking	data
ClaudeImageBlock	image	source
ClaudeDocumentBlock	document	source (via ClaudeDocumentSource hierarchy), title, citations
ClaudeContainerUploadBlock	container_upload	fileId (derived from rawJson)
ClaudeCompactionBlock	compaction	content (server-emitted summary)
ClaudeMCPToolUseBlock	mcp_tool_use	id, name, serverName, input
ClaudeMCPToolResultBlock	mcp_tool_result	toolUseId, content, isError

ClaudeDocumentBlock sends documents to the API. The wire format uses type "document" with a source object whose structure varies by source type; the ClaudeDocumentSource hierarchy covers five shapes:

Class	Source Type	Key Fields
ClaudeBase64DocumentSource	base64	data, mediaType
ClaudeTextDocumentSource	text	data
ClaudeContentDocumentSource	content	content (array of blocks)
ClaudeURLDocumentSource	url	url
ClaudeFileDocumentSource	file	fileId

Four class-side factories on ClaudeDocumentBlock — #fromBase64:, #fromFileId:, #fromText:, #fromURL: — pick the right source subclass so callers rarely touch ClaudeDocumentSource directly; on the response side, ClaudeDocumentSource class>>fromJson: dispatches on the type field and returns the matching concrete source. ClaudeTextBlock's citations collection uses the ClaudeCitation shape described in §27.

Five server-managed blocks appear only in responses and are read-only from a client perspective. The SDK deserializes them so conversations round-trip, but application code does not build them:

Class	Description
`ClaudeServerToolUseBlock`	Server-managed tool invocation (wire type `'server_tool_use'`)
`ClaudeWebSearchResultBlock`	Web search result (wire type `'web_search_tool_result'`)
`ClaudeWebFetchResultBlock`	Web fetch result (wire type `'web_fetch_tool_result'`)
`ClaudeCodeExecutionResultBlock`	Code execution result (wire type `'code_execution_tool_result'`)
`ClaudeBashResultBlock`	Server Bash result (wire type `'bash_code_execution_tool_result'`); fields `toolUseId`, `returnCode`, `stdout`, `stderr`
`ClaudeTextEditorResultBlock`	Server text-editor result (wire type `'text_editor_code_execution_tool_result'`); fields `toolUseId`, `rawJson` (content sub-dict shape varies by editor command: `view` / `create` / `str_replace`)
`ClaudeToolSearchResultBlock`	Tool-discovery result (wire type `'tool_search_tool_result'`); fields `toolUseId`, `toolReferences`

Each server-result block stores its raw JSON payload so fields the server adds in the future survive the round trip.

4.6 Per-Request Options

The durable request controls — metadata, cache control, service tier, beta flags, MCP server list, container, and the streaming mode bit — live on `ClaudeRequestOptions` and are set through the forwarded setters on `ClaudeMessageRequest` (see the Request Parameters tables above). Those options travel with the request on the wire; the SDK never treats them as ad-hoc closures.

A per-call override that doesn't belong on the wire — a timeout bump for one particular long-running request, a debug log block scoped to one call — is a property of the client, not of the request. The preferred idiom is a dedicated `ClaudeClient` configured with the override, used for the one call and then discarded. Clients are cheap and stateless apart from their retry counter, so spinning up a second client with a different timeout is the safe way to scope the change — a shared client never ends up stuck in the overridden state if the call raises:

```
"Dedicated client with a tighter timeout for one slow call."
tightClient := ClaudeClient
    apiKey: client apiKey
    baseUrl: client baseUrl
    timeout: 30.
response := tightClient sendMessage: request.
```

The same pattern is the right shape for a parallel fan-out that wants a per-arm timeout: build one `ClaudeClient` per arm with the timeout baked in, let each arm own its client, and the shared default client is never mutated.

When building a second client is genuinely inconvenient — deep inside a helper that receives the client as an argument, say — mutate the setter inside an `#ensure:` block so the original value is restored even if the call raises:

```
"Ad-hoc override on the shared client, restored on every exit path."
| originalTimeout |
originalTimeout := client timeout.
client timeout: 30.
[ response := client sendMessage: request ]
    ensure: [ client timeout: originalTimeout ].
```

For the canonical first-contact example see §1; for the tokens-only counterpart see §6; for the exception hierarchy every call can raise see §10; for extended thinking and tool use that ride on a Messages request see §23 and §32; for streaming see §21.

Status: *Available*. Anthropic docs: https://docs.claude.com/en/api/messages. Python SDK peer: https://docs.anthropic.com/en/api/messages.

5 Messages Examples

This section is the pattern catalog that accompanies §4 Messages API. Where §4 describes the wire surface and its Pharo bindings, §5 collects the runnable conversation shapes a reader may reach for: multi-turn conversations, conversation replay, and tool-result round-tripping. Every pattern is an ordinary `ClaudeMessageRequest` built through the five builder selectors (`#addUserMessage:`, `#addAssistantMessage:`, `#addAssistantResponse:`, `#addToolResult:content:`, `#addToolResult:content:isError:`); the SDK's job is to serialize that request and return a typed `ClaudeMessage`.

The runnable below is a multi-turn conversation against `ClaudeMockServer`, which makes the pattern testable in the eval server without a live API key or network. The first turn is a user question; the mock returns an assistant answer; the example appends that full response to the request via `#addAssistantResponse:`, adds a follow-up user turn, and sends the request a second time. The accumulated messages collection after both round-trips contains four turns (user, assistant, user, assistant), which is the predicate the example evaluates to:

```
| server client request response |
"Mock the same canned response for both POSTs; the example only checks turn bookkeeping."
server := ClaudeMockServer startOn: 0.
[ server
    respondTo: '/v1/messages'
    with: '{"id":"msg_01","type":"message","role":"assistant","content":[{"type":"text","text
        ":"2 + 2 is 4."}],"model":"claude-sonnet-4-5","stop_reason":"end_turn","stop_sequence":
        null,"usage":{"input_tokens":12,"output_tokens":7,"cache_creation_input_tokens":0,"
        cache_read_input_tokens":0}}'.
  client := ClaudeClient
    apiKey: 'test-key'
    baseUrl: 'http://localhost:' , server port printString
    timeout: 5.
  "Turn 1: build the request and send the first user question."
  request := ClaudeMessageRequest new
    model: ClaudeModel sonnet45;
    maxTokens: 256;
    addUserMessage: 'What is 2 + 2?';
    yourself.
  response := client sendMessage: request.
  "Append the full assistant response; addAssistantResponse: preserves every content block."
  request addAssistantResponse: response.
  "Turn 2: add a follow-up user question and reuse the client."
  request addUserMessage: 'And what is that times 3?'.
  response := client sendMessage: request.
  "The request now has four turns: user, assistant, user, assistant."
  ^ request messages size ]
    ensure: [ server stop ]
"=> 4"
```

`#addAssistantResponse:` takes a whole `ClaudeMessage` and appends it as an assistant turn, calling `#asParam` on the message to produce the request-side shape. Each `ClaudeContentBlock` subclass answers its own `#asParam`, so text, thinking, citations, and redacted-thinking blocks all survive the round trip correctly — preserving redacted thinking is critical when the server's signed prefix would otherwise be lost. `#addAssistantMessage:` is the plain-String variant: when replaying a conversation from a transcript, the reader often has the assistant text as a String and not as a `ClaudeMessage`, and this selector wraps it back into a turn. Tool-use loops use `#addToolResult:content:` (or the `isError:` variant to mark a failed tool call) to pair a result with its tool-use id.

Further patterns — prefill with an assistant turn, vision with an image content block, JSON-schema output, and full tool-use loops — live with the feature sections they exercise: §23 for extended thinking, §25 for vision, §27 for citations, §32 for client-side tool use, §33 for server-executed tools, §29 for JSON schema, and §10 for the error-handling shapes that wrap every call.

Status: *Available*. Anthropic docs: https://docs.claude.com/en/api/messages-examples.

6 Count Tokens

Counting tokens before you send a message lets the caller quote cost up front and gate on context-window headroom without paying for a round-trip that would have exceeded the budget. The count_tokens endpoint accepts the same request shape as `sendMessage:` — the same model, messages, system prompt, and tools — so the count you receive is the same count the billing path would meter a moment later. Applications use this for pre-flight pricing ("this conversation will cost N tokens"), UX warnings when a long turn is approaching the model's input cap, and batch-job gating where you would rather skip a request than overrun a quota.

`ClaudeClient>>countTokens:` sends `POST /v1/messages/count_tokens` and returns a `ClaudeMessageTokensCount`, a value object carrying one field: `inputTokens` (Integer) — the token count for the given request parameters. The response object has no output-token or cached-token breakdown; that information only exists once the model has actually generated a response.

The example below builds a request the same way `sendMessage:` takes one, hands it to `countTokens:`, and reads the count off the result. It is an end-to-end pre-flight check: if `count inputTokens` exceeds the caller's budget, skip the send entirely; otherwise hand the same `params` object to `sendMessage:`.

Count tokens

```
| params count |
"Build a request the same way sendMessage: would."
params := ClaudeMessageRequest new
    model: ClaudeModel opus47;
    maxTokens: 1024;
    addUserMessage: 'How many tokens is this?';
    yourself.

"Send the count-tokens request; surface any API error directly."
[ count := client countTokens: params ]
  on: ClaudeApiError do: [ :e | ^ e signal ].

"count is a ClaudeMessageTokensCount; read the single field."
count inputTokens "=> 14"
```

For the sibling endpoint that actually consumes the counted tokens see §4.

Status: *Available*. Anthropic docs: https://docs.claude.com/en/api/messages-count-tokens. Python SDK peer: https://github.com/anthropics/anthropic-sdk-python#token-counting.

7 Models API — List Models

The Models API tells an application what Claude variants the account can actually call, with their capability limits — useful for model discovery in a UI, for capability gating ("skip this model if `maxInputTokens` is below the prompt I need to send"), and for spotting new aliases when Anthropic publishes a refreshed family. `ClaudeClient>>listModels` sends `GET /v1/models` and returns an `OrderedCollection` of `ClaudeModelInfo`; each entry carries the model ID, display name, token limits, and creation timestamp.

The example below lists every available model and prints its display name alongside its ID — the shape of a discovery UI or a capability audit. `listModels` is synchronous; a paginated variant, `listModels:after:`, returns a `Dictionary` with `data`, `has_more`, `first_id`, and `last_id` when cursor control matters.

List available models

```
| models |
"Fetch every model the account can call; surface API errors directly."
[ models := client listModels ]
  on: ClaudeApiError do: [ :e | ^ e signal ].

"Walk the collection; each entry is a ClaudeModelInfo."
models do: [ :info |
    Transcript show: info displayName, ' (', info id, ')'; cr ].
```

For code that wants to name a model at a call site without hard-coding a string, `ClaudeModel` provides class-side factory methods returning the model ID strings. Applications may also pass any string the API accepts; the constants are conveniences that document which model family a call targets. The factories and the strings they return:

Model constants

```
ClaudeModel haiku35 "=> 'claude-3-haiku-20240307'"
ClaudeModel haiku45 "=> 'claude-haiku-4-5-20251001'"
ClaudeModel sonnet45 "=> 'claude-sonnet-4-5-20250929'"
ClaudeModel sonnet45v "=> 'claude-sonnet-4-5-20250929'"
ClaudeModel sonnet46 "=> 'claude-sonnet-4-6'"
ClaudeModel opus4 "=> 'claude-opus-4-20250514'"
ClaudeModel opus4v "=> 'claude-opus-4-20250514'"
ClaudeModel opus46 "=> 'claude-opus-4-6'"
ClaudeModel opus46v "=> 'claude-opus-4-6'"
ClaudeModel opus47 "=> 'claude-opus-4-7'"
ClaudeModel opus47v "=> 'claude-opus-4-7'"
```

The `v` suffix historically distinguished dated model IDs from floating aliases. In the current table, non-`v` and `v` selectors return the same string for the newer models (`opus46`, `opus47`, `sonnet46`); the suffix exists for future-proofing when Anthropic publishes a dated variant. For older models (`opus4`, `sonnet45`), both forms already resolve to a dated string — there is no functional floating alias.

For the single-model sibling endpoint see §8; for the scope of the Models API surface in this SDK see §3.

Status: *Available*. Anthropic docs: https://docs.claude.com/en/api/models-list. Python SDK peer: https://github.com/anthropics/anthropic-sdk-python#list-models.

8 Models API — Get a Model

When an application already knows a model ID and needs only that model's capability details — maximum input tokens, maximum output tokens, the human-readable display name — fetching the single record is cheaper than listing the catalog and filtering. `ClaudeClient>>getModel:` sends `GET /v1/models/{id}` and returns a single `ClaudeModelInfo`; an unknown ID raises `ClaudeNotFoundError` (404), which a capability-gating path typically treats as "model unavailable" rather than a hard failure.

The example below retrieves one model by ID, handles the unknown-ID case with the typed 404, and then reads the capability fields the application cares about. The `on: ClaudeNotFoundError` idiom is the right shape for a capability check: other errors (network, 5xx, auth) propagate.

Get specific model info

```
| info |
"Retrieve capability details for a specific model;
```

```
 catch 404 as 'model unavailable' and let other errors propagate."
[ info := client getModel: 'claude-sonnet-4-5' ]
  on: ClaudeNotFoundError do: [ :e |
    Transcript show: 'Unknown model'; cr. ^ nil ].

"Read the capability fields off the ClaudeModelInfo response."
info maxInputTokens "=> 200000"
info maxTokens "=> 8192"
```

`ClaudeModelInfo` is the response type for both `listModels` and `getModel:`. It carries six fields:

Variable	Type	Description
`id`	String	Model identifier
`displayName`	String	Human-readable name
`maxInputTokens`	Integer	Maximum input context length
`maxTokens`	Integer	Maximum output tokens
`type`	String	Always `'model'`
`createdAt`	String	ISO-8601 creation timestamp

For the list endpoint that returns an `OrderedCollection` of these records see §7.

Status: *Available*. Anthropic docs: https://docs.claude.com/en/api/models. Python SDK peer: https://github.com/anthropics/anthropic-sdk-python#retrieve-model.

9 Handling Stop Reasons

Every Messages response carries a `stopReason` — a Symbol that tells the caller why Claude stopped generating. Applications dispatch on it to continue a tool-use loop, resume a paused turn, surface a natural completion to the user, or fall through to telemetry when the model ran into a limit. The stop reason is the first thing an agent reads off a response; it is how the agent decides what to do next.

The wire protocol actually uses two value domains and the SDK preserves them. The top-level `stop_reason` field from the response is parsed by `ClaudeStopDetails` class¿¿`fromJson:` and surfaced as `ClaudeMessage>>stopReason`, a Symbol. The separate `stop_details.type` sub-object (currently used for the `'refusal'` category with its own `category` and `explanation` fields) is accessed through `ClaudeMessage>>stopDetails` and the `category` / `explanation` accessors — it does *not* appear as a `stopReason` value. The table below covers the five Symbol values the `stop_reason` field actually emits:

stopReason	Meaning
`#end_turn`	Natural completion
`#max_tokens`	Hit `maxTokens` cap or model max
`#stop_sequence`	Hit a `stopSequences` entry
`#tool_use`	Model invoked one or more tools
`#pause_turn`	Long-running turn paused; resume with the same messages

Refusals arrive as `stopReason = #end_turn` plus a populated `stopDetails` sub-object whose `category` names the safety category (for example, `'violence'`) and whose `explanation` carries a short rationale. Code that needs to distinguish a refusal from an ordinary natural completion reads `response stopDetails category` after checking `stopReason = #end_turn` — both paths use the same `#end_turn` stop reason on the wire.

Applications that need to branch on `stopReason` match on the Symbol directly — there is no wrapper class or visitor pattern. A typical first-contact dispatch covers the three branches an agent loop actually

cares about: complete response, tool-use continuation, and paused-turn resumption. The runnable below starts with a nil-guard: `ClaudeMessage>>stopReason` returns nil when `stopDetails` has not been populated (an in-flight streaming response, a manually constructed message), and silent nil-dispatch is not a shape any caller wants:

Dispatch on stopReason

```
| response reason |
response := client sendMessage: request.
reason := response stopReason.

"Nil-guard: an incomplete response has no stopReason yet."
reason ifNil: [ ^ nil ].

"Natural completion: hand the text back to the caller."
reason = #end_turn ifTrue: [ ^ response textContent ].

"Tool use: the model invoked tools; run them and loop."
reason = #tool_use ifTrue: [ ^ self handleToolUse: response ].

"Paused turn: long-running work suspended; resume with same messages."
reason = #pause_turn ifTrue: [ ^ self resumeWithSameMessages ].

"Fall-through: #max_tokens or #stop_sequence --- telemetry / surface."
^ self recordUnexpectedStop: reason
```

For the refusal case — and for other non-2xx failures that never produce a response in the first place — see §10; for the `#tool_use` branch and how a tool-use loop continues see §32; for the `#pause_turn` branch and the Extended Thinking flow that drives it see §23.

Status: *Available*. Anthropic docs: https://docs.claude.com/en/api/handling-stop-reasons.

10 Errors

The Messaging SDK turns every non-2xx HTTP response from the Messages API into a typed Smalltalk exception. Rather than carrying a status-code field on a single error class, the SDK dispatches at the response boundary: a 401 becomes a `ClaudeAuthError`, a 429 becomes a `ClaudeRateLimitError`, a 529 becomes a `ClaudeOverloadedError`. Callers write `[ ... ] on: ClaudeAuthError do: [:e | ...]` and let everything else propagate — the idiomatic Smalltalk analog of the Python SDK's `APIStatusError` hierarchy.

`ClaudeApiError` is the abstract base, a subclass of `Error`. Ten concrete subclasses cover the status codes the API actually returns:

Class	HTTP Status	Retryable	When
`ClaudeApiError`	(abstract)	—	Base class for all API errors
`ClaudeInvalidRequestError`	400	no	Malformed or invalid request
`ClaudeAuthError`	401	no	Invalid API key
`ClaudeBillingError`	402	no	Billing issue
`ClaudePermissionError`	403	no	Insufficient permissions
`ClaudeNotFoundError`	404	no	Unknown model or endpoint
`ClaudeTimeoutError`	408 / 504	yes	Request or gateway timeout
`ClaudeRateLimitError`	429	yes	Rate limit exceeded; honors `Retry-After`
`ClaudeServerError`	500	yes	Internal server error
`ClaudeOverloadedError`	529	yes	API overloaded (Anthropic-specific)
`ClaudeNetworkError`	599 (synthetic)	no	Connection refused, TLS handshake, DNS — a refused connection stays refused

Every instance carries the status code, the Anthropic error message, the request ID from the `request-id` response header, the raw JSON body for logging, and an optional `retryAfter` parsed from the `retry-after` header on 429 responses. The class-side factory `fromStatusCode:json:requestId:` is what `ClaudeClient` calls internally; in application code you almost never construct these directly — you catch them.

The example below demonstrates the dispatch end-to-end. It calls the factory with a 401 status code, signals the resulting exception, and shows a handler that catches the specific subclass (`ClaudeAuthError`) the factory routed to. The same shape is what `ClaudeClient` produces on a real auth failure, so the application code that handles it looks identical:

```
| caught |
caught := [
    "Build the error and raise it (the SDK does this internally for any non-2xx response)."
    (ClaudeApiError
        fromStatusCode: 401
        json: '{"error":{"message":"bad key"}}'
        requestId: 'req_1') signal ]
    on: ClaudeAuthError
    do: [ :e | e class name ]. "Handler catches the specific subclass."
caught
"=> #ClaudeAuthError"
```

Streaming responses get the same exception hierarchy. The Messages API uses Server-Sent Events for incremental token delivery (see §21); an in-flight stream can deliver an SSE `error` event after the response has started — a server-side overload, a model timeout, a 529 spike. The SDK routes those events through `fromSseErrorData:`, populating the synthetic status code from the Anthropic `error.type` string so `isRetryable` and `printOn:` behave identically whether the error arrived on the status line or in a streaming payload. One `on: ClaudeApiError do: [...]` handler covers both paths.

The SDK retries the retryable failures (the *Retryable* column above) automatically — five retries by default, with exponential backoff from 0.5 s to 8 s and ±25% jitter. When the response carries a `Retry-After-Ms` or `Retry-After` header, the SDK uses the header value instead of the computed delay, clamped to 30 s to defend against pathological values. Tests that exercise the non-retry path must explicitly set `maxRetries: 0` or they will wait out the full backoff sequence; calls from the UI thread must be wrapped in a forked Process because the backoff delay blocks the current process.

Two divergences from the Python SDK are worth knowing. HTTP 529 is Anthropic-specific and not IANA-assigned — the SDK treats it as a retryable 5xx. HTTP 599 is synthetic; the Python SDK raises `APIConnectionError` without a status code, while the Pharo SDK keeps everything inside the `ClaudeApiError` hierarchy so a single `on: ClaudeApiError do: [...]` catches both wire-protocol errors and connection errors uniformly.

For the rate-limit monitoring headers surfaced on `ClaudeRateLimitError` see §13; for how mid-stream SSE errors are routed see §21.

Status: *Available*. Anthropic docs: https://docs.claude.com/en/api/errors. Python SDK peer: https://github.com/anthropics/anthropic-sdk-python#handling-errors.

11 API Versions

Every request to the Messages API carries an `anthropic-version` header that pins the wire format. Anthropic guarantees that requests sent with a given version keep the same request and response shape indefinitely, so a client that pins a version keeps working across Anthropic's platform changes until it opts into a newer one. The baseline is `2023-06-01`, which is what every first-party SDK ships with; new versions land when Anthropic changes response shapes or introduces incompatible defaults.

In Pharo the version lives on `ClaudeClient`'s `apiVersion` slot, initialized to `'2023-06-01'` on construction and sent as the `anthropic-version` header by every HTTP-client factory (`newHttpClient`,

newFileHttpClient, newSkillHttpClient). Override it with apiVersion: on a per-client basis when you need to opt into a new wire version during migration.

This section is reference material; no runnable example applies.

Status: *Available*. Anthropic docs: https://docs.claude.com/en/api/versioning.

12 Beta Headers

Some Messages API features sit behind an anthropic-beta header that gates access to not-yet-stable surface: new input shapes (PDFs, extended-output models), new endpoints (Files API, Skills API, MCP client), or experimental response semantics. Each beta is a string identifier like pdfs-2024-09-25 or files-api-2025-04-14; callers can opt into several at once by sending a comma-joined header, and Anthropic lists the active identifiers on its beta-headers documentation page.

Pharo handles beta headers two ways, depending on whether the beta is an SDK implementation detail or an application-level choice. ClaudeClient owns the automatic path: newFileHttpClient sends the Files API beta on every Files request and newSkillHttpClient sends the Skills beta on every Skills request, so Files and Skills callers never touch beta headers themselves. For per-request betas — the MCP client, PDF support, extended-output models — callers set betas: on ClaudeRequestOptions (directly, or via ClaudeMessageRequest which delegates). At send time, newHttpClientFor: reads the collection off the request options, joins it with commas, and emits it as the anthropic-beta header.

The SDK ships a typed catalog of the known beta identifiers as ClaudeBetaHeader, a class intended for class-side use. Class-side messages return the wire String: ClaudeBetaHeader pdfs answers 'pdfs-2024-09-25', ClaudeBetaHeader structuredOutputs answers 'structured-outputs-2025-11-13', and so on. Typos are surfaced when the send is evaluated (as an unknown selector / doesNotUnderstand:) instead of landing only as a server 400, and ClaudeBetaHeader allKnown returns the full list as an OrderedCollection for docs generation, validation, and test parity. The catalog is pure discovery: ClaudeRequestOptions>>betas: still accepts an arbitrary collection of Strings, so unknown-to-the-SDK identifiers remain usable without waiting on a catalog update.

The example below demonstrates the opt-in path in both spellings. It builds a request options object, sets two beta identifiers, and asks it for the header value that ClaudeClient would emit on the wire — the same value that newHttpClientFor: would place on the anthropic-beta header. The first body matches ClaudeRequestOptions's class-comment doctest byte-for-byte; the second shows the catalog-aware form and produces the identical wire value:

```
"Build an options object with two active betas and read back the wire value."
ClaudeRequestOptions new
    betas: #('pdfs-2024-09-25' 'output-128k-2025-02-19');
    betaHeader
"=> 'pdfs-2024-09-25,output-128k-2025-02-19'"
```

```
"Same two betas via the typed catalog -- identical wire value."
ClaudeRequestOptions new
    betas: { ClaudeBetaHeader pdfs. ClaudeBetaHeader output128k };
    betaHeader
"=> 'pdfs-2024-09-25,output-128k-2025-02-19'"
```

To use this in an application, call betas: directly on the ClaudeMessageRequest before sendMessage: — the request owns a ClaudeRequestOptions instance from construction and the convenience setter writes through to it. For the less common cases that betas: does not cover, mutate request requestOptions directly. The Files and Skills APIs require no such wiring — their beta headers are part of the endpoint's identity and the SDK adds them automatically via ClaudeBetaHeader filesApi and ClaudeBetaHeader skills.

Status: *Available*. Anthropic docs: https://docs.claude.com/en/api/beta-headers.

13 Rate Limits

Anthropic enforces rate limits per API key on three axes: requests per minute (RPM), input tokens per minute (ITPM), and output tokens per minute (OTPM). The limits scale with the account tier, and every response carries monitoring headers (`anthropic-ratelimit-requests-remaining`, `anthropic-ratelimit-tokens-remaining`, and companions) that tell the caller how close they are to the next 429. When a limit is hit, the API returns a 429 with a `retry-after` hint; the Pharo SDK surfaces this as `ClaudeRateLimitError` (§10) and honors the hint automatically on retry.

For applications that make many concurrent requests on the same API key, reactive 429 handling is not enough — the retry-after delay blocks the calling process, and several callers racing to retry can stampede the budget right back to zero. The SDK ships `ClaudeRateLimiter`, a token-bucket limiter that coordinates request rates across every `ClaudeClient` sharing a key. The bucket starts full (five tokens by default, one per permitted burst request), drains one token per call to `acquire`, and refills at `requestsPerMinute / 60` tokens per second. When the bucket is empty, callers block in `acquire` until a token becomes available or the 30-second timeout fires.

The example below demonstrates constructing a limiter at a chosen rate. The body matches `ClaudeRateLimiter`'s class-comment doctest byte-for-byte — the printed form is the limiter's own `printOn:`, which reports its configured RPM and burst size:

```
"Build a limiter sized for 50 requests per minute and read its printed form."
ClaudeRateLimiter new requestsPerMinute: 50
"=> a ClaudeRateLimiter(50 rpm, 5 burst)"
```

To use the limiter with a client, attach it via `rateLimiter:` on `ClaudeClient`; `sendMessage:` then calls `acquire` before dispatching each HTTP request, blocking when the bucket runs dry. The class-side factories `forKey:` and `forKey:model:` return a shared limiter keyed on an API key (and optionally the model name), so every client built against the same credentials automatically contends on the same bucket — the default model-aware factory picks 20 RPM for Opus, 50 RPM for Haiku, and 40 RPM elsewhere.

For reactive observation of the live limit state, every `sendMessage:` response carries a typed `ClaudeResponseMetadata` attached as `ClaudeMessage>>responseMetadata` (with a `rateLimits` convenience reader for shorter call chains). The metadata holds eleven nil-tolerant slots: `requestId` echoed from the `request-id` header, the three-axis `anthropic-ratelimit-*` counters (`rateLimitRequestsLimit`, `rateLimitRequestsRemaining`, `rateLimitRequestsReset`, `rateLimitInputTokensLimit`, `rateLimitInputTokensRemaining`, `rateLimitInputTokensReset`, `rateLimitOutputTokensLimit`, `rateLimitOutputTokensRemaining`, `rateLimitOutputTokensReset`), and the optional `retryAfter` hint that 429 responses set per RFC 7231 §7.1.3. The integer counters parse via `asInteger`; reset timestamps stay as the raw ISO-8601 String the server sends so callers can parse on demand without losing precision. The `hasRateLimits` predicate answers `true` when any of the nine ratelimit slots is populated; `requestId` and `retryAfter` never count toward it. The same metadata attaches to `ClaudeApiError` subclasses on the failure path so `ClaudeRateLimitError` handlers can read live counters and the reset timestamp to pick a smarter backoff than `Retry-After` alone.

```
"Read the live ratelimit counters off a successful send."
| client request response |
client := ClaudeSDKExampleSupport resolveClient.
request := ClaudeMessageRequest new
    model: ClaudeModel sonnet46;
    maxTokens: 64;
    addUserMessage: 'ping';
    yourself.
response := client sendMessage: request.
{ response rateLimits rateLimitRequestsRemaining.
  response rateLimits rateLimitRequestsReset.
  response rateLimits hasRateLimits }
"=> #(98 '2026-04-22T17:30:00Z' true) (values vary per response)"
```

For the rate-limit exception that 429 responses produce see §10; for service-tier interactions with rate limits see §14.

Status: *Available*. Anthropic docs: https://docs.claude.com/en/api/rate-limits.

14 Service Tiers

Service tiers let the caller trade throughput against cost and latency for a given request. The Messages API exposes two values on the `service_tier` request-body field: `auto` (the default — the scheduler picks the best tier available to the account, including priority when capacity permits) and `standard_only` (force the standard tier even when priority capacity is available, typically for cost-predictability). Asynchronous batch processing is not a `service_tier` value; it is reached via a separate endpoint (§16) whose requests are billed at the batch discount regardless of what the per-request `service_tier` field would otherwise select. Anthropic returns the tier actually applied on the response so callers can observe which tier the scheduler chose.

In Pharo the tier is set via `serviceTier:` on `ClaudeMessageRequest`, which delegates to the underlying `ClaudeRequestOptions`. The setter takes the wire string directly — pass either a raw String or one of the `ClaudeServiceTier` catalog constants (`ClaudeServiceTier auto`, `ClaudeServiceTier standardOnly`) which return those exact strings. The catalog is a discovery surface, not a typed enum: the setter still accepts arbitrary Strings so unknown values flow through to Anthropic if it ever extends the field. Validation is deliberately left to the server because client-side enforcement would lock out forward-compat tiers. The request serializer emits the field only when a value is set, so requests that leave the tier unset fall back to Anthropic's default (equivalent to `'auto'`).

The example below demonstrates selecting the default tier explicitly on a message request and reading the configured value back. It uses the `ClaudeMessageRequest` convenience setter, which writes through to the request options; the body matches `ClaudeMessagingRequestOptionsExample`'s canonical build shape:

```
"Build a request that asks the scheduler to pick the best tier."
| request |
request := ClaudeMessageRequest new
    model: ClaudeModel sonnet46;
    maxTokens: 1024;
    serviceTier: 'auto'; "Let the scheduler pick; 'standard_only' forces standard."
    addUserMessage: 'What is the capital of France?';
    yourself.
request serviceTier
"=> 'auto'"
```

The tier that Anthropic actually applied to a response is read off `ClaudeUsage>>serviceTier`. The server echoes the chosen tier in the response body under `usage.service_tier` and `ClaudeUsage class >> fromJson:` parses that field into the `serviceTier` slot; the value is nil when Anthropic does not echo the field (older endpoints, or count-tokens replies). Note that the response-side vocabulary is distinct from the request-side `ClaudeServiceTier` catalog: the request says *which scheduling policy to use* (`'auto'` or `'standard_only'` today), while `usage.service_tier` reports *which backend tier was actually applied* (typically `'standard'`, `'priority'`, or `'batch'`). Treat the `usage.service_tier` value as a raw String for now; `ClaudeServiceTier` does not enumerate response-side identifiers, only request-side policies. The broader rate-limit telemetry that the live response carries lives on `ClaudeMessage>>responseMetadata` (see §13), so callers reading the applied tier and the rate-limit counters drill into two adjacent objects on the same response rather than re-parsing headers themselves.

```
"Read the tier the scheduler applied to a response."
| client request response |
client := ClaudeSDKExampleSupport resolveClient.
request := ClaudeMessageRequest new
    model: ClaudeModel sonnet46;
    maxTokens: 64;
    serviceTier: ClaudeServiceTier auto;
    addUserMessage: 'ping';
    yourself.
```

```
response := client sendMessage: request.
{ response usage serviceTier. ClaudeServiceTier allKnown asArray }
"=> #('standard' #('auto' 'standard_only')) (applied value varies)"
```

For the Batches API that owns asynchronous batch-tier processing see §16; for the rate-limit interactions the `auto` and `standard_only` tiers share see §13.

Status: *Available*. Anthropic docs: https://docs.claude.com/en/api/service-tiers.

15 Client SDKs

Anthropic publishes first-party client SDKs in Python, TypeScript, Java, Ruby, C#, PHP, and a CLI, and maintains a catalog of additional community and partner SDKs. Each SDK wraps the same Messages API over HTTP and exposes the same response shapes; they differ in idiom, packaging, and what their host ecosystem lets them express. The Pharo SDK this specification describes is a Punt Labs port into the Pharo Smalltalk ecosystem, not Anthropic-maintained; it ships in two layers, a Messaging layer (this document) and an Agent layer (separate specification).

This section is reference material; no runnable example applies.

Status: *Available*. Anthropic docs: https://docs.claude.com/en/api/client-sdks.

16 Batches API

The Batches API accepts up to 100 000 Messages-API requests in a single job (256 MB per batch) and runs them asynchronously against the `batch` service tier at a discounted per-token rate. Callers submit a batch, poll for completion, and then stream the per-request results as a single download once the job finishes. The endpoint covers six operations: create, retrieve, retrieve results, list, cancel, and delete. Applications that can tolerate minutes of latency in exchange for the batch-tier discount use this surface to run overnight evaluations, bulk classification jobs, or warehouse backfills against the same Messages API they use interactively.

The Pharo SDK does not implement the Batches API today. No `ClaudeBatch*` classes exist in any `Claude-Messaging-*` package, and `ClaudeClient` exposes no batch selectors. The implementation plan when it lands mirrors the Files API shape: a typed request/response layer in `Claude-Messaging-Types`, `ClaudeClient` selectors for the six operations, and a streaming reader for the results download so large batches do not materialize in full in VM memory.

This section is reference material; no runnable example applies.

Status: *Not yet implemented*. Anthropic docs: https://docs.claude.com/en/api/creating-message-batches.

17 Files API

The Files API uploads a file once and lets later requests reference it by ID instead of inlining base64 content on every call — you save tokens, you save bandwidth, and large PDFs or images stop bloating every request payload. The Pharo surface covers six Files API operations via seven selectors on `ClaudeClient` (`uploadFile:` and `listFiles` each have a one-arg and a parameterized form), each mirroring one of Anthropic's `client.beta.files.*` endpoints in the Python SDK. The endpoints all sit behind the `files-api-2025-04-14` beta header, which the SDK injects automatically; callers never set it.

Selector	HTTP	Returns
`uploadFile:`	POST `/v1/files`	`ClaudeFileMetadata`. One-arg form infers filename and MIME type from the `FileReference`.
`uploadFile:filename:mimeType:`	POST `/v1/files`	`ClaudeFileMetadata`. Three-arg form to override either.
`listFiles`, `listFiles:`	GET `/v1/files`	`ClaudeFilePage`. Pass a `ClaudeFileListParams` for cursor pagination (`limit`, `afterId`, `beforeId`).
`getFileMetadata:`	GET `/v1/files/{id}`	`ClaudeFileMetadata`
`downloadFile:`	GET `/v1/files/{id}/content`	A `ByteArray`. Only Claude-generated files are downloadable; user uploads report `downloadable = false`.
`deleteFile:`	DELETE `/v1/files/{id}`	`ClaudeDeletedFile`

The four typed request and response classes carry the API's data shape: `ClaudeFileMetadata` (id, filename, mimeType, sizeBytes, createdAt, downloadable), `ClaudeFilePage` (data array, hasMore, firstId, lastId), `ClaudeFileListParams` (limit, afterId, beforeId), and `ClaudeDeletedFile` (id, deleted).

The most common usage is upload-once-reference-many: you upload a PDF or image, get a file ID back, and then attach it to message content blocks via `ClaudeDocumentBlock fromFileId:` or the equivalent on `ClaudeImageBlock`.

The example below demonstrates the read path against `ClaudeMockServer` so it runs in the eval server without an API key or network connection. It registers two canned responses (the file inventory and a single file's metadata), points a `ClaudeClient` at the mock's local port, calls `listFiles` and `getFileMetadata:`, and extracts four fields from the typed response objects. The same body lives in `ClaudeClient`'s class comment as the canonical doctest:

```
| server client meta page |
"Spin up an in-image mock HTTP server so the example needs neither network nor API key."
server := ClaudeMockServer startOn: 0.
[ "Register the canned response for GET /v1/files (the inventory listing)."
  server
    respondTo: '/v1/files'
    with: '{"data":[{"id":"file-abc","type":"file","filename":"notes.txt","mime_type":"text/
        plain","size_bytes":6,"created_at":"2026-04-21T00:00:00Z","downloadable":false}],"
        has_more":false,"first_id":"file-abc","last_id":"file-abc"}'.
  "Register the canned response for GET /v1/files/file-abc (single-file metadata)."
  server
    respondTo: '/v1/files/file-abc'
    with: '{"id":"file-abc","type":"file","filename":"notes.txt","mime_type":"text/plain","
        size_bytes":6,"created_at":"2026-04-21T00:00:00Z","downloadable":false}'.
  "Build a ClaudeClient that talks to the mock instead of api.anthropic.com."
  client := ClaudeClient
    apiKey: 'test-key'
    baseUrl: 'http://localhost:' , server port printString
    timeout: 5.
  "Exercise the two read selectors and pull four fields out of the typed responses."
  page := client listFiles.
  meta := client getFileMetadata: 'file-abc'.
  ^ { page data size. page data first id. meta filename. meta sizeBytes } ]
    ensure: [ server stop ] "Always stop the mock, even on error."
"=> #(1 'file-abc' 'notes.txt' 6)"
```

Real upload, download, and delete usage is mechanically the same shape — swap the mock server for a production client (`ClaudeSDKExampleSupport resolveClient` pulls the API key from the OS keyring or `ANTHROPIC_API_KEY` and points at `api.anthropic.com`) and swap the verbs. `client uploadFile: aFileReference` returns a metadata object whose `id` is the file ID; `client downloadFile: aFileId`

returns a raw `ByteArray`; `client deleteFile: aFileId` returns a `ClaudeDeletedFile` whose `deleted` field reflects success.

Two implementation details are worth knowing. Uploads stream from disk in 64 KB chunks via `ClaudeStreamingMultiPartEntity`, so VM memory usage is bounded by the chunk size plus framing rather than by the file size — gigabyte uploads are safe. Downloads follow the server's `downloadable` flag: only Claude-generated files (tool outputs, skill artifacts) come back; your own uploads are listable and referenceable but not readable. Unlike the Skills API's hard 25 MB per-file and 100 MB total caps, the Files API has no client-side size limit — whatever the server accepts goes through.

For attaching uploaded files to message content see §4.5; for the sibling Skills API that shares the streaming multipart machinery see §18; for the exception hierarchy that every Files call can raise see §10.

Status: *Available*. Anthropic docs: https://docs.claude.com/en/api/files-create. Python SDK peer: https://docs.anthropic.com/en/api/files-list.

18 Skills API

The Skills API packages a directory tree as a reusable, versioned artifact the model can load into a code-execution container on a Messages request. A skill is a folder rooted at `SKILL.md` plus whatever supporting files the skill's scripts reference; uploading that folder gives back a skill id and a version id the model can mount by reference on any future turn. The endpoint ships behind the `skills-2025-10-02` beta header; every skill URL also appends `?beta=true` inline, matching Anthropic's hosted-beta path convention (pagination requests follow the query string with `&page=`).

The Pharo surface splits across two packages by role. Request-side specs that bind into the Messages container — `ClaudeContainerParams` (the container shape) and `ClaudeSkillSpec` (a single skill reference) — live in `Claude-Messaging-Types` alongside `ClaudeRequestOptions`, because that is where a caller reaches for them when building a request. Metadata responses, pagination wrappers, list-query params, and deletion receipts live in `Claude-Messaging-Skills`.

Class	Role
`ClaudeSkill`	Skill metadata response (id, display title, source, latest version)
`ClaudeSkillVersion`	Versioned skill metadata response (id, name, description, directory, version)
`ClaudeDeletedSkill`	Skill deletion response
`ClaudeDeletedSkillVersion`	Version deletion response
`ClaudeSkillPage`	Page-token-paginated list of skills
`ClaudeSkillVersionPage`	Page-token-paginated list of versions
`ClaudeSkillListParams`	List query (page, source filter, limit)
`ClaudeSkillVersionListParams`	Version list query (page, limit)
`ClaudeContainerParams`	Container request shape (id + skills array, in `Claude-Messaging-Types`)
`ClaudeSkillSpec`	Skill ref for container loading (`skill_id` + `type` + `version`, in `Claude-Messaging-Types`)

Both `ClaudeContainerParams` and `ClaudeSkillSpec` carry an `#extraFields` slot so a `fromJson:/asJson` roundtrip preserves any wire keys Anthropic ships in the future; this mirrors the forward-compat pattern `ClaudeMCPServerDefinition` established for the MCP connector, and means a consumer can read today's shape today without losing tomorrow's fields. The beta header itself is hardcoded on the Skills HTTP client — `#newSkillHttpClient` emits `anthropic-beta: skills-2025-10-02` unconditionally, so callers do not opt in via `betas:`, the same shape `#newFileHttpClient` uses for the Files API.

Uploads stream from disk in 64 KB chunks via `ClaudeStreamingMultiPartEntity`; even a skill at the 100 MB total limit never materializes in VM memory. The multipart body uses multiple `files[]` parts (the literal bracketed field name is what the server expects — an unbracketed `files` field name is rejected), one part per file under the directory. The shared helper `#buildStreamingSkillPartsInto:fromDirectory:` assembles those parts for both `#uploadDirectory:displayTitle:` (new skill) and `#uploadVersionFor:fromDirectory:` (new version of an existing skill). By contrast, `#uploadFile:filename:mimeType:` (Files API, §17) drives `ClaudeStreamingMultiPartEntity` directly without a shared helper — the two APIs diverge enough in shape that a cross-API helper would cost more than it saves. Filenames in the multipart parts are prefixed with the upload directory's basename and use POSIX (forward-slash) separators, so a file at `skillDir/SKILL.md` serializes as `<basename>/SKILL.md` and a nested `skillDir/helpers/util.py` as `<basename>/helpers/util.py`; the server requires every uploaded file to live under a single top-level folder named after the skill.

Skills list endpoints return page-token pagination (`?page=` request, `next_page` response field) rather than the Files API's cursor-id shape. `ClaudeSkillPage` and `ClaudeSkillVersionPage` both expose a `#nextPage` slot (String or nil) and a derived `#hasMore` predicate, so a pager loop reads as a `[ page hasMore ] whileTrue:` driving the next `#page:` on the list-params.

On the container side, `ClaudeRequestOptions` accepts either shape through its polymorphic `#container:` setter — a plain String container id (back-compat with the original Code Execution Tool) or a `ClaudeContainerParams` instance holding an id plus a skills array. The dispatch happens in `#addFieldsTo:`, which emits the String verbatim when `isString` holds and calls `asJson` on the params otherwise; existing String-input callers keep working unchanged. On the response side, `ClaudeExecutionContainer` carries a `#skills` slot whose entries parse back as `ClaudeSkillSpec` instances — the in-container response shape is identical to the request-side spec (`skill_id`, `type`, `version`), so one class serves both.

Before a single byte leaves the client, `#uploadDirectory:displayTitle:` (via `#buildStreamingSkillPartsInto:fromDirectory:`) enforces three preconditions and raises a plain `Error` with a pinned message format for each, so a caller sees a clear precondition failure instead of a 400 round-trip. The guards fire in a fixed order: first, `SKILL.md` must exist at the upload directory root; second, the directory basename must match the `name:` field in `SKILL.md`'s YAML frontmatter, which the guard parses line-by-line inside the leading `---` … `---` block (no YAML parser dependency); third, every file must be under `ClaudeClient #maxSkillFileBytes` (25 MB = 26,214,400 bytes) and the summed upload under `#maxSkillUploadBytes` (100 MB = 104,857,600 bytes). A skill whose directory name disagrees with the frontmatter AND whose files are oversized reports the name mismatch first; the precedence matters because the caller usually wants to know about the cheaper fix before the more expensive one.

Deletion is two-phase by server contract: `#deleteSkill:` rejects a skill that still has versions, so the caller must enumerate via `#listVersionsFor:` and call `#deleteVersion:for:` on each before the skill itself can go. The SDK does not ship a cascade helper in v1.0; a follow-up bead tracks adding one if the two-step dance turns out to be a papercut in practice.

Skills API

```
"Upload a skill directory; returns a ClaudeSkill metadata response"
| client skill request |
client := ClaudeClient apiKey: 'sk-ant-...'.
skill := client
    uploadDirectory: 'skills/python-data-analysis' asFileReference
    displayTitle: 'Python data analysis'.

"Build a Messages request that loads the skill into its container"
request := ClaudeMessageRequest new
    model: ClaudeModel sonnet46;
    maxTokens: 1024;
    "Polymorphic container: either a String id or ClaudeContainerParams"
    container: (ClaudeContainerParams new
        skills: { ClaudeSkillSpec new
            skillId: skill id;
```

```
        type: 'custom';
        "Explicitly pin the current published version; omit version: to let Anthropic use the
             latest published version"
        version: skill latestVersion;
        yourself };
    yourself);
addUserMessage: 'Use the python-data-analysis skill to plot ...';
yourself.
```

For the sibling Files API that shares `ClaudeStreamingMultiPartEntity` see §17; for the feature-guide companion that frames when to reach for a skill versus a tool see §35; for the Messages request the container binds into see §4.

Status: *Available*. Anthropic docs: https://docs.claude.com/en/api/skills-guide.

19 Prompt Tools API

Prompt Tools is Anthropic's set of helper endpoints for developing and improving prompts programmatically: `generate` drafts a first-pass prompt from a task description, `templatize` extracts variables from a concrete prompt, and `improve` proposes revisions to an existing prompt. The intended use case is prompt-engineering automation — CI-driven A/B workflows, batch prompt iteration, agent build pipelines that evolve their own prompts — rather than per-turn production traffic. The endpoints live under `/v1/experimental/`, which is Anthropic's signal that the shape may change before graduating to stable.

The Pharo SDK does not ship Prompt Tools in v1.0. The experimental path is the reason: a typed Smalltalk surface built against `/v1/experimental/` today would likely require a breaking revision when Anthropic moves the endpoints out of that path, and the SDK's v1.0 promise is to freeze the public types we ship. `ClaudeClient` exposes no `generatePrompt:`, `templatizePrompt:`, or `improvePrompt:` selectors, and no request/response classes exist for the three endpoints. The Phase 2 plan is to add the typed surface when the endpoints leave `/v1/experimental/`; until then, teams that need these capabilities call the endpoints directly via `ZnClient` against the raw URL. This section is reference material; no runnable example applies.

For the Messages API that the generated and improved prompts ultimately feed into see §4.

Status: *Not yet implemented*. Anthropic docs: https://docs.claude.com/en/api/prompt-tools-generate.

20 Admin API

The Admin API is Anthropic's operator-facing control plane: it manages organizations, workspaces, members, and API keys; it reports Usage and Cost; it exposes Claude Code Analytics. The audience is a platform operator auditing spend, provisioning keys for new teams, attributing cost across workspaces, or pulling analytics into a monitoring stack. Requests are gated by a distinct *Admin API key*, separate from the standard API key the Messages surface uses, and minted from the admin console rather than the developer console.

The Pharo SDK does not ship the Admin API in v1.0. The scope declaration in §1 targets the standard messaging surface — send, stream, tool use, files, skills — because that is what an agent developer reaches for; the Admin surface is a different product with a different key and a different user. `ClaudeClient` exposes no admin selectors, and no organization/workspace/member/key-management classes exist in the Messaging-layer packages. The Phase 2 plan, when user demand materializes, is likely a separate `ClaudeAdminClient` (or an opt-in admin surface activated with the Admin API key) rather than overloading `ClaudeClient`; the two key types and the two user journeys are distinct enough

that blending them into one client would be more confusing than helpful. This is operator surface, not developer surface. This section is reference material; no runnable example applies.

For the scope declaration that tracks SDK-level omissions see §1.

Status: *Not yet implemented*. Anthropic docs: https://docs.claude.com/en/api/administration-api.

Part III

Build with Claude

21 Streaming Messages

Streaming delivers a response token-by-token as the model generates it, rather than forcing the caller to wait for the full completion. The transport is Server-Sent Events: the same `POST /v1/messages` endpoint as §4, just with `stream: true` on the request. Applications reach for streaming whenever a user is watching the response (chat UIs, IDE assistants, dashboards) so partial output shows up immediately instead of after a several-second round-trip.

The Pharo SDK exposes two modes. The block-callback form `#streamMessage:do:` is the idiomatic path for most applications: it opens the stream, reads events into typed dictionaries, hands each event to the caller's block, and returns the fully accumulated `ClaudeMessage` when the stream closes. The raw-socket form `#openStreamingSocket:` is the lower-level path for callers that need to interleave reads with other work (a `SharedQueue` producer/consumer pattern, a forked UI updater, etc.); it returns the socket for the caller to drive directly through `ClaudeSSEDecoder`. The runnable example below uses `#streamMessage:do:` and is the primary shape a first-time reader should try. It is the canonical doctest from `ClaudeClient`'s class comment:

```
| client request final |
client := ClaudeSDKExampleSupport resolveClient.
request := ClaudeMessageRequest new
        model: ClaudeModel opus46;
        maxTokens: 1024;
        addUserMessage: 'Count to five.';
        yourself.
final := client streamMessage: request do: [ :event |
        (event at: 'event') = 'content_block_delta' ifTrue: [
                | delta |
                delta := (event at: 'data') at: 'delta'.
                Transcript show: (delta at: 'text' ifAbsent: [ '' ]) ] ].
Transcript cr; show: 'stop reason: ', final stopReason.
```

Each `event` the block receives is a Dictionary with two String keys: `'event'` is the SSE event-type String (`'message_start'`, `'content_block_delta'`, ...), and `'data'` is the parsed-JSON Dictionary payload. The block runs synchronously inside `#streamMessage:do:`; when the stream closes, `#streamMessage:do:` returns the accumulated `ClaudeMessage` with every content block finalized, the stop reason set, and the token-usage counts populated — the same shape a non-streaming `#sendMessage:` would have returned.

21.1 SSE Protocol

The streaming API returns Server-Sent Events (SSE) — a text protocol where each event is a pair of lines (event type, JSON data) separated from the next event by a blank line. Event types identify what happened (message started, block appended, delta arrived); the data payload carries the structured details. A minimal SSE framing looks like:

```
event: message_start
data: {"type":"message_start","message":{...}}

event: content_block_delta
data: {"type":"content_block_delta","delta":{"type":"text_delta","text":"Hello"}}

event: message_stop
data: {"type":"message_stop"}
```

21.2 Event Types

The eight event types the SDK recognizes map one-to-one to the steps `ClaudeMessage` walks while assembling a response from deltas. `message_start` delivers the empty envelope (id, role, model, initial usage counts); `content_block_start` announces a new content block (text, tool use, thinking, . . .) with its index; `content_block_delta` carries the incremental payload keyed on the delta subtype; `content_block_stop` finalizes the block so streaming accumulators can parse any partial-JSON buffers; `message_delta` updates stop reason, stop sequence, and the final usage counters; `message_stop` closes the stream; `ping` is a heartbeat the SDK ignores silently; `error` carries an API error that the client translates into a typed `ClaudeApiError` subclass:

Event	Payload
`message_start`	Envelope for an empty `ClaudeMessage` (id, role, model, initial usage)
`content_block_start`	Content block type and index; creates the right `ClaudeContentBlock` subclass
`content_block_delta`	Incremental text, input JSON, thinking, signature, or citation
`content_block_stop`	Block index (finalization signal)
`message_delta`	Stop reason, stop sequence, final output-token count
`message_stop`	End of stream
`ping`	Heartbeat (ignored)
`error`	Error during streaming; raised via `ClaudeApiError fromSseErrorData:`

The `content_block_delta` payload is polymorphic, discriminated by its own `delta.type` field. Five delta subtypes cover every incremental update the server emits; each maps to a specific field on the active content block:

Delta Type	Field	Description
`text_delta`	`text`	Incremental text content
`input_json_delta`	`partial_json`	Incremental tool-input JSON (parsed at `content_block_stop`)
`thinking_delta`	`thinking`	Incremental extended-thinking text
`signature_delta`	`signature`	Thinking block signature
`citations_delta`	`citation`	Citation reference appended to the active text block

21.3 SSE Decoder

`ClaudeSSEDecoder` parses raw SSE text into typed event dictionaries and supports two calling shapes matching the two modes above. The buffered shape — `#parse:do:` — takes a complete SSE response body as a String and calls the caller's two-argument block once per event with `(eventType, dataDictionary)`. Because the full body is received before the first callback fires, buffered mode is simpler but loses the latency win. Callers that already have the raw body (for example, after `#streamRawBody:`) use it when they just want to iterate the event stream post hoc:

```
| body |
"Fetch the full SSE body as a String, then iterate the events."
body := client streamRawBody: request.
ClaudeSSEDecoder new
    parse: body
    do: [ :eventType :data |
        "eventType is a String; data is a Dictionary (or nil on unparseable)."
        self handleEvent: eventType data: data ].
```

The incremental shape — `#onStream:` plus `#hasNext` and `#nextEvent` — reads lines off an underlying stream until a blank line completes an event, then returns the assembled Dictionary. This is the shape a `SharedQueue` producer uses so a consumer process can handle events as they arrive while the producer reads the socket. The standard producer/consumer pattern pairs `#openStreamingSocket:` with `#ClaudeSSEDecoder onStream:` and a `SharedQueue`; the producer forks at `userBackgroundPriority`, the consumer defers UI updates to the Morphic thread via `WorldState defer:`:

```
| queue process message |
queue := SharedQueue new.
message := ClaudeMessage new.

"Producer: read the TLS socket in a background process."
process := [ | socket decoder |
  socket := client openStreamingSocket: request.
  decoder := ClaudeSSEDecoder onStream: socket.
  [ decoder hasNext ] whileTrue: [
    queue nextPut: decoder nextEvent ].
  queue nextPut: #end ]
    forkAt: Processor userBackgroundPriority
    named: 'Claude SSE reader'.

"Consumer: process events as they land on the queue."
[ | event |
  event := queue next.
  event == #end ] whileFalse: [
    message accumulate: event.
    "Push partial content to the UI on the Morphic thread."
    WorldState defer: [
      textMorph contents: message textContent ] ].
```

`#openStreamingSocket:` returns a `ClaudeStreamingSocket` when the client's `baseUrl` is an HTTPS endpoint: it opens a TCP socket, wraps it in `ZdcSecureSocketStream` with Server Name Indication, sends the HTTP/1.1 POST request (stream mode true, every required header set), reads the response status line and headers, and verifies a 2xx status — non-2xx raises a typed `ClaudeApiError`. When the `baseUrl` is an HTTP URL (mock servers in tests), the same selector returns a `ReadStream` over the buffered body so tests don't need TLS infrastructure. Both return types are compatible with the decoder's `#atEnd` / `#nextLine` protocol. Per-read timeout on the TLS socket comes from `streamingTimeout` on `ClaudeClient`, default 300s. The decoder itself exposes a small protocol for callers that drive it directly:

Method	Returns	Behavior
`#onStream:`	`ClaudeSSEDecoder`	Class-side factory. Wraps any object answering `#atEnd` / `#nextLine`. Initializes the line buffer and event accumulator.
`#hasNext`	Boolean	Reads lines from the stream until a complete event is assembled (blank line) or the stream ends. Returns `true` when an event is ready, `false` at end-of-stream. Blocks on the socket read — must run in a background Process when the stream is a live socket.
`#nextEvent`	Dictionary	Returns the last assembled event as `Dictionary('event' -> type. 'data' -> dict)`. Call only after `#hasNext` returned `true`.
`#close`	—	Closes the underlying stream.

Line parsing follows the SSE specification: a line beginning with `event:` sets the event type for the pending event, a line beginning with `data:` appends to the data buffer, a blank line dispatches, and a line beginning with `:` is a comment and is ignored. `#streamMessage:do:` wraps the decoder loop in a `ConnectionTimedOut` handler that re-raises as `ClaudeTimeoutError` (HTTP 408, retryable at the caller level) and a `NetworkError` handler that re-raises as `ClaudeNetworkError` (synthetic 599, non-retryable); callers that drive the decoder directly through `#openStreamingSocket:` handle the same two exception classes themselves. To cancel an in-flight stream, terminate the producer process — the socket is closed on process termination:

```
"Kill the background reader; the socket is closed during termination."
process terminate.
```

21.4 Message Accumulator

`ClaudeMessage>>accumulate:` builds a complete message from a sequence of streaming events and is what `#streamMessage:do:` calls under the hood to assemble the return value. The method reads the `'event'` key off each event dictionary and dispatches to a private handler per event type: `message_start` initializes id, type, turn, role, model, and usage; `content_block_start` instantiates the matching `ClaudeContentBlock` subclass via the type registry and appends it to the active turn's content; `content_block_delta` sends `#applyDelta:` to the last block, which dispatches polymorphically on the delta subtype (`text_delta` concatenates into `text`, `input_json_delta` buffers into `partial_json`, `thinking_delta` and `signature_delta` concatenate into `thinking`/`signature`, `citations_delta` appends one `ClaudeCitation` to the active text block's citations); `content_block_stop` calls `#finalizeAfterStreaming` on the last block so tool-use/MCP-tool-use blocks can parse their accumulated partial-JSON buffer into a proper Dictionary; `message_delta` merges the final stop reason, stop sequence, and updated output-token count; `error` raises the matching `ClaudeApiError` subclass via `ClaudeApiError class>>fromSseErrorData:`.

The net effect is that after `#streamMessage:do:` returns, the accumulated `ClaudeMessage` is indistinguishable from the `#sendMessage:` response you would have gotten had you called the non-streaming variant — same content blocks, same stop reason, same usage counts. The Pharo-specific threading model for streaming (forking the producer process, deferring to the Morphic thread) lives in the companion *Pharo-Specific Implementation Notes* under the Pharo Threading Model section.

For the non-streaming `#sendMessage:` counterpart and the full Messages API surface see §4; for how mid-stream SSE errors are routed into the typed exception hierarchy see §10; for extended thinking — which delivers its own `thinking_delta` and `signature_delta` events alongside the normal text deltas — see §23.

Status: *Available*. Anthropic docs: https://docs.claude.com/en/docs/build-with-claude/streaming. Python SDK peer: https://docs.anthropic.com/en/api/messages-streaming.

22 Prompt Caching

Mark a text block, tool definition, or tool-result block as cacheable, and Anthropic caches the serialized prefix across requests sharing the same API key — subsequent requests pay a fraction of the input-token cost on whatever's cached. The Pharo primitive is `ClaudeCacheControl`, a small value object that serializes to `{"type":"ephemeral"}` with an optional `"ttl"` of `'5m'` or `'1h'`. The on-wire shape is identical to the Python SDK's `cache_control={"type": "ephemeral"}` block parameter; the Pharo class just wraps it in a typed object you can pass around and inspect.

The class-side factories build a marker; `asJson` converts it to the wire-format Dictionary. The default is the five-minute ephemeral cache; `ephemeralWithTTL:` sets a custom TTL (Anthropic currently accepts only `'5m'` and `'1h'`). Both forms appear below as their own runnable expressions:

```
"Build a default-TTL marker and serialize it for the wire."
ClaudeCacheControl ephemeral asJson
"=> an OrderedDictionary('type'->'ephemeral')"

"Build a marker with an explicit five-minute TTL."
(ClaudeCacheControl ephemeralWithTTL: '5m') asJson
"=> an OrderedDictionary('type'->'ephemeral' 'ttl'->'5m')"
```

To use prompt caching in an application, mark a content block as cacheable and send the request through `ClaudeClient` as you would any other. The most common pattern caches a long system prompt or RAG prefix. `ClaudeMessageRequest`'s `systemBlocks:` setter takes a collection of `ClaudeTextBlock` instances, one or more of which can carry a cache marker:

```
| client request response |
client := ClaudeClient apiKey: 'sk-ant-...'.

request := ClaudeMessageRequest new
    model: ClaudeModel sonnet46;
    maxTokens: 1024;
    systemBlocks: {
        (ClaudeTextBlock text: 'You are a helpful assistant. <long reusable instructions...>')
            cacheControl: ClaudeCacheControl ephemeral };
    addUserMessage: 'What is the capital of France?';
    yourself.

response := client sendMessage: request.
response usage cacheCreationInputTokens. "tokens written to cache (first-call miss)"
response usage cacheReadInputTokens. "tokens served from cache (subsequent hits)"
```

The first call pays full input cost on the marked prefix and writes it to the cache (`cacheCreationInputTokens`); subsequent calls within the TTL pay a fraction (`cacheReadInputTokens` reflects what the cache served). Both counters live on `ClaudeUsage`, which is on every Messages response — no extra API call is needed to observe cache effectiveness.

Caching tool definitions and tool-result prefixes follows the same shape, but the wiring details differ. The SDK exposes six `cacheControl:` setters across six classes (`ClaudeMessageRequest`'s is a convenience that delegates to `ClaudeRequestOptions`), split into two groups depending on *when* the marker gets serialized. `ClaudeTextBlock` and `ClaudeServerTool` take the `ClaudeCacheControl` instance directly — they call `asJson` on it themselves at request-serialization time. `ClaudeTool`, `ClaudeToolResultBlock`, `ClaudeRequestOptions`, and `ClaudeMessageRequest` (which delegates to `ClaudeRequestOptions`) take the already-serialized Dictionary — you call `asJson` yourself before passing it in. The split is deliberate: the second group's `cacheControl:` setters predate the typed wrapper, and tightening them would break callers that already pass raw dictionaries. The Pharo type system doesn't catch the mismatch, so the rule of thumb is: `ClaudeTextBlock` and `ClaudeServerTool` take an instance; everything else takes `... asJson`.

```
"Cache a tool definition (raw-Dictionary group: pass ... asJson)"
| tool |
```

```
tool := ClaudeTool
    name: 'analyze'
    description: 'Analyze code'
    inputSchema: ClaudeToolInputSchema new.
tool cacheControl: ClaudeCacheControl ephemeral asJson.

"Cache a tool-result prefix that the model already saw on the previous turn"
(ClaudeToolResultBlock toolUseId: 'tu_123' content: 'result')
    cacheControl: (ClaudeCacheControl ephemeralWithTTL: '5m') asJson.
```

Three constraints are worth planning around. Anthropic enforces a maximum of four `cache_control` markers per request server-side, so put them on the largest static prefixes first — the system prompt, the tool definitions, the RAG context. TTL values currently support only `'5m'` (five minutes) and `'1h'` (one hour); the SDK keeps them as strings so a future addition like `'24h'` would need no code change. And the only `type` Anthropic exposes is `ephemeral`; a future `persistent` type would land as a new class-side factory (`ClaudeCacheControl persistent`) rather than a new instance variable.

For the content-block classes that carry cache markers see §4.5; for `ClaudeTool` and `ClaudeServerTool` see §32; for where `ClaudeUsage`'s cache fields surface on the response envelope see §4.

Status: *Available*. Anthropic docs: https://docs.claude.com/en/docs/build-with-claude/prompt-caching. Python SDK peer: https://docs.anthropic.com/en/docs/build-with-claude/prompt-caching.

23 Extended Thinking

Extended thinking asks Claude to reason before answering and returns that reasoning as a separate content-block type the application can display, hide, or audit. An agent that writes code, solves a multi-step math problem, or synthesizes a long document benefits from visible chain-of-thought: the thinking blocks carry the intermediate steps, and the final `ClaudeTextBlock` carries the answer. Applications that surface reasoning to end users (tutoring, code-review assistants, evaluation harnesses) read the thinking blocks alongside the answer; applications that only need the answer discard them. The feature is opt-in per request and controlled by a token budget — small budgets produce no thinking, large budgets produce detailed reasoning, and the model chooses how much of the budget to spend.

The Pharo surface has three pieces. `ClaudeThinkingParams` configures the request: `enabled:` takes an Integer budget in tokens (required), `adaptive` lets Anthropic size the budget, and `disabled` turns thinking off. `ClaudeMessageRequest` accepts the thinking configuration through its `thinking:` setter, which takes a Dictionary — pass the params object's `asDictionary` form. On the response, `ClaudeThinkingBlock` subclasses `ClaudeContentBlock` with an `isThinkingBlock` predicate and `thinking` accessor; redacted reasoning (which Anthropic occasionally replaces for safety) arrives as the sibling `ClaudeRedactedThinkingBlock` type carrying opaque `data` the application must round-trip verbatim without inspection.

The runnable below enables thinking with a 1024-token budget, sends the request, and filters the response content for the thinking block. Anthropic requires `max_tokens > budget_tokens`, so `maxTokens` is set to 2048 to leave room for both the reasoning budget and the final answer:

```
| client request response thinkingBlock |
client := ClaudeClient apiKey: 'sk-ant-...'.

"Enable thinking with a 1024-token reasoning budget. The factory
 returns a ClaudeThinkingParams instance; asDictionary converts it
 to the wire-format Dictionary that thinking: expects."
request := ClaudeMessageRequest new
    model: ClaudeModel sonnet46;
    maxTokens: 2048;
    thinking: (ClaudeThinkingParams enabled: 1024) asDictionary;
    addUserMessage: 'What is the Monty Hall problem? Show your reasoning, then give the final
        answer.';
```

```
    yourself.

response := client sendMessage: request.

"Response content is an OrderedCollection of ClaudeContentBlock.
 detect:ifNone: picks the first thinking block; nil-guard the lookup
 because the model may skip thinking on simple prompts even when
 enabled."
thinkingBlock := response content
    detect: [ :b | b isThinkingBlock ]
    ifNone: [ nil ].
thinkingBlock ifNotNil: [ :b |
    Transcript show: b thinking; cr ].

"The final answer is still in the text block; textContent joins all
 text blocks in response order."
response textContent
```

Three details shape how an application uses the feature. Budget tuning ships today: `budgetTokens` is a real instance variable that `enabled:` sets directly, and a budget below roughly 1024 tokens often yields no thinking block at all because the model decides the problem does not warrant deliberation. Larger budgets give the model more room to reason but cost more input-equivalent tokens and extend response latency; the idiomatic starting point is 1024. The three modes serve different workloads: `enabled:` pins a fixed budget (predictable latency and cost), `adaptive` lets Anthropic size the budget to the prompt (simpler to configure), and `disabled` turns thinking off when a previously-enabled request template is being reused for a quick query. And the adaptive mode accepts an optional display hint via `adaptiveWithDisplay:` (`'summarized'` or `'omitted'`) controlling what the UI receives.

Two further pieces shape multi-turn agents that consume thinking. The first is the signature round-trip. Every `ClaudeThinkingBlock` returned by the API carries an opaque cryptographic `signature` the server computes from the thinking content plus server-side secrets. When an application replays a prior assistant turn back to the model — the standard shape of multi-turn tool-use loops — the signature must arrive at the server byte-for-byte unchanged or the request is rejected. The SDK preserves the signature automatically: `asJson` on a `ClaudeThinkingBlock` emits the `signature` field, and `ClaudeMessageRequest>>asJson` carries it through the serialized assistant turn. Applications writing thinking-aware agents get the round-trip for free; the discipline is pinned by `ClaudeThinkingBlockTest>>testSignatureRoundTrip` (the wire path) and `ClaudeThinkingBlockTest>>testStonSignatureRoundTrip` (the persisted-session path used by repository-backed conversations).

The second is interleaved thinking. By default the model emits at most one thinking block per response, before the final text answer. With the `interleaved-thinking-2025-05-14` beta the model may emit a thinking block in *every* assistant turn — including the turns that wrap individual tool calls. Tool-use-heavy agents benefit because the model can reason about each tool result in turn, choose the next call deliberately, and reason between calls rather than only at the end of the response. Whether that intermediate reasoning reaches the end user is governed separately by `adaptiveWithDisplay:` earlier in this section (`'summarized'` or `'omitted'`). Opt in per request:

```
request betas: { ClaudeBetaHeader interleavedThinking }.
```

The runnable `ClaudeInterleavedThinkingExample class>>buildRequest` constructs a multi-turn conversation where two prior assistant turns each carry their own thinking block plus a tool-use block, with tool results between them. The request is structural: its thinking blocks carry placeholder signatures chosen for clarity in the doctest, and those placeholders must be replaced with real signatures captured from a prior live response before the request can be sent successfully to the API. The example still doubles as a test fixture: the placeholder signatures round-trip through the wire serializer, demonstrating exactly the discipline the previous paragraph describes.

For `ClaudeMessageRequest`'s other request knobs and the in-flight request object see §4; for how thinking interacts with tool calls see §32.

Status: *Available*. Anthropic docs: https://docs.claude.com/en/docs/build-with-claude/extended-thinking.

24 Context Editing

Context editing is a server-side set of strategies for trimming long conversations to fit within Claude's context window. Anthropic exposes it as a `context_management` request parameter with named strategies: `clear_tool_uses_20250919` drops tool-call and tool-result pairs older than a threshold, `clear_thinking_20251015` drops thinking blocks from earlier turns. Agents that run long tool-use loops accumulate thousands of tokens of obsolete tool output each turn; a multi-day conversation retaining every thinking block rapidly exhausts the window. Context editing lets the server drop the old material without the client rebuilding the conversation history locally — cheaper than shuttling every turn through a summarizer and more principled than unbounded growth.

The feature is opted in via the `context-management-2025-06-27` beta header (see §12) plus the `context_management` request parameter naming one or more strategies and their thresholds. Anthropic's docs describe the current strategy set; the set expands as new editing passes land.

The Pharo SDK does not implement context editing today. `ClaudeRequestOptions` carries no `context_management` slot, `ClaudeMessageRequest` exposes no setter for it, and the request serializer does not emit the field. When implemented, Pharo will expose a typed `ClaudeContextManagement` class with enumerated strategies and a `contextManagement:` setter on `ClaudeRequestOptions` matching the shape of `ClaudeThinkingParams` / `thinking:`. This section is reference material; no runnable example applies.

For the beta header machinery that will carry the opt-in see §12; for the scope declaration that tracks SDK-level omissions see §3.

Status: *Not yet implemented*. Anthropic docs: https://docs.claude.com/en/docs/build-with-claude/context-editing.

25 Vision / Multimodal

Vision lets Claude see images. The user turn carries a content block whose type is `image` alongside any text blocks; Claude processes the image in its visual encoder and reasons about it the same way it reasons about text. Applications use vision for chart and screenshot analysis, document OCR, photograph description, UI testing, and any workflow where the input arrives as pixels rather than characters. Multiple images per turn are supported, and images mix freely with text blocks in the same request.

Pharo wires vision through `ClaudeImageBlock`, which carries a `source` Dictionary shaped for the wire. Three class-side factories cover the three source types. `fromUrl:` points at a publicly-accessible image URL; Anthropic fetches and processes the image server-side. `base64:mediaType:` carries the encoded bytes inline; nothing leaves the client beyond the request itself, which matters for images the application generates or derives from private data. `fromFileId:` references an image uploaded earlier through the Files API (§17); the server looks up the bytes by ID, avoiding a re-upload on every request when the same image is used across a long conversation. Two convenience factories build on `base64:mediaType:`: `fromFile:` reads a local image file and infers the media type from the extension, and `fromForm:` captures a Pharo `Form` (e.g., a screenshot from `World imageForm`) as a PNG.

The runnable below uses `fromUrl:` because it reads cleanest. The image block and a text-block prompt are composed into a single user turn via `addUserContent:`, which accepts a collection of `ClaudeContentBlock` instances in arbitrary order:

```
| client image request response |
client := ClaudeClient apiKey: 'sk-ant-...'.
```

```
"Build the image block from a public URL. Anthropic will fetch and
 process the image server-side; no local IO is required."
image := ClaudeImageBlock fromUrl: 'https://upload.wikimedia.org/wikipedia/commons/4/47/
    PNG_transparency_demonstration_1.png'.

request := ClaudeMessageRequest new
    model: ClaudeModel sonnet46;
    maxTokens: 1024;
    "addUserContent: returns the added ClaudeMessage, not the request.
     The cascade's yourself targets the original receiver (the request),
     so the assignment captures the request object correctly."
    addUserContent: { (ClaudeTextBlock text: 'Describe this image in one sentence.'). image };
    yourself.

response := client sendMessage: request.
response textContent
```

The three factories choose based on where the image lives. Public URL is simplest when the image already sits on a CDN or public host, costs no client-side bandwidth, and Anthropic's fetcher handles the download. Base64 inline is the right default for images the application generates on the fly or derives from private sources the caller does not want to publish to a public URL — the bytes travel in the request body and never leave the client's control boundary. File-ID source is the right default for large images reused across many requests: upload once via the Files API, pay the upload cost once, and reference the ID on every subsequent request. Both `fromURL:` and `fromUrl:` are accepted across `ClaudeImageBlock` and `ClaudeDocumentBlock` (§26); either casing builds the same wire shape.

The inline-source factories validate the raw byte count against Anthropic's 5 MB per-image cap before encoding, so oversized inputs fail fast with a `ClaudeInvalidRequestError` rather than waiting for the server's 400 response. URL and file-ID sources skip client-side validation — the server validates at fetch or lookup time.

Supported formats are PNG, JPEG, GIF, and WebP; Anthropic's docs carry the current per-image size and dimension limits and the per-request image count cap.

For the content-block wire shape see §4.5; for the Files API used by `fromFileId:` see §17; for the sibling document block that handles PDFs see §26.

Status: *Available*. Anthropic docs: https://docs.claude.com/en/docs/build-with-claude/vision.

26 PDF / Document Block

PDF support lets Claude read a PDF inline the same way vision lets it read images. Anthropic's server decodes the PDF, extracts text, figures, and tables, and feeds the material into the model's context alongside the rest of the user turn. Applications use it for contract review, research-paper Q&A, financial-report summarization, and any workflow where the source document is a PDF rather than markdown or plain text. Short PDFs ride inline in the request body; large or frequently-reused PDFs upload once to the Files API and reference by ID on every subsequent request. Anthropic's docs cite a rough budget of 1500–3000 input tokens per page.

Pharo represents documents through `ClaudeDocumentBlock`, whose `source` slot holds a `ClaudeDocumentSource` subclass matching the wire-format source type. Four class-side factories cover the four source types. `fromFileId:` references a PDF uploaded earlier through the Files API (§17) — the production-idiomatic path for large or reused PDFs because it avoids re-uploading the bytes on every request. `fromBase64:` carries the PDF bytes inline as a base64 string — simplest for one-shot short PDFs where the upload round-trip would add more overhead than it saves. `fromText:` wraps already-extracted plain text in a document block — useful when upstream code has done its own PDF parsing and wants the text treated as a document (for citations; see §27) rather than as an inline user message. `fromURL:` points at a publicly-accessible PDF; `fromUrl:` works as an alias for callers that prefer the lowercase convention (§25 covers the same bidirectional alias on `ClaudeImageBlock`).

Applications that need to estimate request cost before sending a large PDF can call `tokensPerPageRange` on `ClaudeDocumentBlock`, which returns an `Interval` spanning Anthropic's rough 1500–3000 per-page budget. Multiply by the page count for a minimum and maximum input-token estimate, or call `ClaudeClient>>countTokens:` for an exact count on the assembled request.

The runnable below uses `fromBase64:` because it runs end-to-end without a prior Files-API upload. The document block and a text-block prompt compose into a single user turn the same way vision's image block does. For production use with a large or reused PDF, swap `fromBase64:` for `fromFileId:` with the identifier returned by `ClaudeClient>>uploadFile:`:

```
| client doc request response |
client := ClaudeClient apiKey: 'sk-ant-...'.

"Carry the PDF bytes inline as a base64 string. The fixture below is
 a minimal 1-page PDF (615 bytes raw, 820 base64 chars) lifted byte-
 for-byte from ClaudeMessagingPDFExample>>examplePDFBase64. No beta
 header is required for base64-inline PDFs."
doc := ClaudeDocumentBlock fromBase64: ClaudeMessagingPDFExample examplePDFBase64.

request := ClaudeMessageRequest new
    model: ClaudeModel sonnet46;
    maxTokens: 1024;
    "addUserContent: returns the added message; the cascade's
     yourself targets the request so the assignment captures it."
    addUserContent: { (ClaudeTextBlock text: 'Summarize the contents of this PDF in one sentence
        .'). doc };
    yourself.

response := client sendMessage: request.
response textContent
```

Four source choices map to four setup costs. `fromFileId:` amortizes a single upload across many requests and is the right default when the same PDF will be referenced repeatedly (multi-turn Q&A over a fixed document). `fromBase64:` skips the Files API entirely — shortest code path, right when the PDF is short and used once. `fromText:` is the escape hatch when the caller already has extracted text and wants document-aware handling (which currently matters chiefly for citations; non-extracted PDFs are far richer because Anthropic's server extracts figures and tables too). `fromURL:` is the right default when the PDF lives on a public host the model should fetch directly. URL-source PDFs require the `document-url-2025-04-15` beta header on the request; the base64 and file-ID paths have no beta dependency.

For the Files API uploader see §17; for the sibling image block see §25; for combining PDFs with citations see §27.

Status: *Available*. Anthropic docs: https://docs.claude.com/en/docs/build-with-claude/pdf-support.

27 Citations

Citations give the application character-indexed pointers back to the source spans Claude used to produce its answer. When a user turn carries a citation-enabled document, each assistant text block on the response arrives with a `citations` array naming the document and the start and end character offsets of the cited excerpt, plus the excerpt text itself. Applications use citations for RAG grounding (the UI can link each sentence of the answer back to its source), audit trails (a reviewer can verify that every claim traces to the input), and compliance workflows where the answer must cite its evidence. Unlike most Claude behaviour, citation content is uniquely server-computed: the cited excerpt is extracted from the raw document bytes the client sent, and the character offsets are assigned by the server's citation pipeline. A client cannot fabricate a citation.

The Pharo surface has two halves. On the request, the caller toggles citations on a `ClaudeDocumentBlock` with the typed trio `enableCitations`, `disableCitations`, and the `citationsEnabled` predicate. Under the hood the enable slot holds the wire-shape Dictionary `{ #enabled -> true } asDictionary` (mirroring `"citations": {"enabled": true}` on the JSON side); the typed methods exist so callers do not have to spell that shape by hand, and `disableCitations` sets the slot to `nil` so `asJson` omits the key entirely — the API recognizes key omission as the opt-out, not an explicit `{ enabled: false }`. The raw `citations:` setter remains available as an escape hatch for any future wire-shape additions. Only documents carrying the enable marker are eligible for citation; asking about a plain text block produces no citations even when the answer is quoted verbatim. On the response, each `ClaudeTextBlock` may carry a `citations` OrderedCollection of `ClaudeCitation` instances; the accessors are `citedText` (the extracted excerpt), `documentIndex` (the zero-based position of the cited document in the user turn's content array), `documentTitle` (the document's `title` field when set), `startCharIndex`, and `endCharIndex`.

The runnable below attaches a citations-enabled document to the user turn, sends the request, and iterates the response text blocks to surface every citation:

```
| client doc request response citedBlocks |
client := ClaudeClient apiKey: 'sk-ant-...'.

"Build the document block, set an optional title (which appears in
 citations as documentTitle), and enable citations. The typed method
 enableCitations sets the underlying slot to the wire-shape Dictionary
 { #enabled -> true } asDictionary; callers who need to emit a
 future wire extension can still reach for the raw citations: setter."
doc := ClaudeDocumentBlock fromText: 'Pharo images save with open sockets. Restoring creates
    stale half-open descriptors. The preferred recovery path is to rebuild from clean Tonel
    source.'.
doc title: 'Pharo Image Discipline'.
doc enableCitations.

request := ClaudeMessageRequest new
    model: ClaudeModel sonnet46;
    maxTokens: 1024;
    "addUserContent: returns the added message; yourself targets the
     request receiver so the assignment captures it."
    addUserContent: { (ClaudeTextBlock text: 'What is the recovery path for a saved image with
        stale sockets?'). doc };
    yourself.

response := client sendMessage: request.

"Filter response text blocks to the ones carrying citations and
 surface each citation with its excerpt and character range."
citedBlocks := response content select: [ :b |
    b isTextBlock and: [ b citations notNil and: [ b citations isNotEmpty ] ] ].
citedBlocks do: [ :b |
    b citations do: [ :c |
        Transcript
            show: '[', (c documentTitle ifNil: [ '?' ]), ' ',
                c startCharIndex printString, '-', c endCharIndex printString,
                ']: ', c citedText;
            cr ] ]
```

Two design details shape how applications use the feature. The enable marker lives on the document block, not the request, because a user turn can mix citation-enabled documents with non-citable text content — per-block granularity lets the caller decide which sources the model may cite. And the citation objects on the response point at character offsets in the original document bytes the client sent, which is what lets the UI highlight the exact span: the application holds the authoritative document and the citation tells it where to look. Prompts that ask about a specific quoted claim ("what does the document say about X") yield richer citations than open-ended summaries, because the model has a clear target to cite.

For the document block and its four source factories see §26; for citations on image-derived OCR see §25; for uploading the source documents through the Files API see §17.

Status: *Available*. Anthropic docs: https://docs.claude.com/en/docs/build-with-claude/citations.

28 Search Results

Search results are a GA content-block type that Anthropic emits when a RAG document has been searched: each block wraps an excerpt the model retrieved and carries its own `citations` array tying the excerpt back to character offsets in the source. Applications use them to expose typed RAG responses where each retrieved snippet already comes citation-linked — like §27 Citations, but inlined into a dedicated block type rather than attached to free-form text. The right time to reach for search results is when the application runs a RAG pipeline with automatic citation wiring; sending raw documents and enabling citations on them (the §27 path) stays the right choice when the application supplies the documents itself and leaves retrieval to the model.

The Pharo SDK does not implement the search-result content block today. No `ClaudeSearchResultBlock` class exists in any `Claude-Messaging-*` package, no `search_result` entry is registered in the `ClaudeContentBlock` type registry, and the block never decodes on the response — an unrecognised type currently falls through to the raw-JSON preservation path rather than being materialised as a typed instance. The server-tool sibling `ClaudeWebSearchResultBlock` is a different class and a different feature: it is the result block the built-in `web_search` tool emits (see §33), whereas the RAG `search_result` block is a first-class content-block type the model may produce directly. Phase 2 adds a typed `ClaudeSearchResultBlock` class, a `search_result` entry in the content-block type registry, and a worked example demonstrating how the block's internal `citations` array surfaces through the response.

This section is reference material; no runnable example applies.

For the response-side citation wire shape that the feature shares see §27; for the server-tool web-search sibling see §33.

Status: *Not yet implemented*. Anthropic docs: https://docs.claude.com/en/docs/build-with-claude/search-results.

29 Structured Outputs

Structured outputs constrain the model's response to validate against a JSON Schema the caller provides. The model is guaranteed — not merely prompted — to emit a document whose fields match the schema's property names, value types, and required keys, which lets the application parse the response as structured data without retry loops or validation layers on top. Applications use the feature for form filling (extract named fields from a user utterance into a known record type), information extraction (normalize one shape into another across heterogeneous inputs), and API glue (treat the model's output as a typed payload the next service can consume directly). Opt in by adding the `structured-outputs-2025-11-13` beta to the request and by supplying a format of shape `{type: "json_schema", schema: {...}}` — when both are present the server enforces the schema during generation. Anthropic's public documentation names the wire parameter `output_format`, but the Pharo SDK's serializer (and the request body it produces) nests the format inside an `output_config` object, so the on-the-wire JSON field is `output_config.format` carrying the `{type, schema}` value. The two are the same feature under two names; the Pharo SDK emits the grouped form because the same `output_config` object also carries the reasoning `effort` hint described later in this section.

Pharo exposes the feature through the `ClaudeOutputFormat` hierarchy and two request-side setters on `ClaudeMessageRequest`: `outputFormat:` takes a `ClaudeOutputFormat` instance (the public API), and `effort:` takes the optional reasoning-effort hint (`'low'`, `'medium'`, `'high'`, `'xhigh'`, or `'max'`). Internally the request aggregates both into a `ClaudeOutputConfig` value object held on the private

outputConfig slot; callers reach the format and effort through the two public setters rather than assigning the config directly, which keeps the request API symmetric with other parameter groups (sampling params, thinking params) that also sit behind flat accessors on the request.

Two concrete format classes subclass ClaudeOutputFormat. ClaudeTextOutputFormat is the default plain-text output and is client-side only: its asJson returns nil so the serializer omits output_config from the wire payload entirely (equivalently, it omits what Anthropic's docs call the output_format parameter), and its encoding (#ascii, #latin1, #utf8) and markup (#plain, #microdown, #markdown, #json, #html, #latex) axes translate into a system-prompt suffix rather than into a wire field. ClaudeJsonSchemaOutputFormat is the schema-constrained variant — it wraps a ClaudeToolInputSchema (the same builder used for tool definitions, see §32) and serializes to {"type": "json_schema", "schema": {...}}. Construct it via ClaudeJsonSchemaOutputFormat class>>schema: directly, or via the convenience factory ClaudeOutputFormat class>>jsonSchema: which delegates to the same class.

The runnable below builds a two-field customer-order schema, hands it to outputFormat: on the request, and sends the message through ClaudeClient. The response text then parses as a JSON document matching the schema:

```
| client schema request response |
client := ClaudeClient apiKey: 'sk-ant-...'.

"Build the JSON schema. ClaudeToolInputSchema is the same builder used
 for tool definitions; addProperty:type:description: registers each
 field and require: marks it as mandatory in the emitted schema."
schema := ClaudeToolInputSchema new
    addProperty: 'name' type: 'string' description: 'Customer name';
    addProperty: 'amount' type: 'number' description: 'Order total';
    require: 'name';
    require: 'amount';
    yourself.

"Construct the request. betas: opts into the structured-outputs beta;
 outputFormat: is the public setter that routes the format through
 the internal ClaudeOutputConfig. The broken outputConfig: setter
 some older snippets reference does not exist --- use outputFormat:."
request := ClaudeMessageRequest new
    model: ClaudeModel opus47;
    maxTokens: 1024;
    betas: #('structured-outputs-2025-11-13');
    outputFormat: (ClaudeJsonSchemaOutputFormat schema: schema);
    addUserMessage: 'Record a $42.50 coffee order for Marie Curie as JSON.';
    yourself.

"Send and parse. textContent is the assistant's JSON payload as a String;
 STONJSON fromString: decodes it into a Dictionary whose 'name' value
 is a String and whose 'amount' value is a Number, matching the schema."
response := client sendMessage: request.
STONJSON fromString: response textContent
```

The reasoning-effort hint is orthogonal to the output format. Set it via request effort: 'high' on any request that supports it; the two live in the same ClaudeOutputConfig aggregator but surface as independent levers on the request.

Three worked schema patterns cover most of the shapes a real extraction request needs. The first is the *enum-constrained string field*, used when a value must come from a fixed vocabulary — order status, currency code, severity level. The builder method #addProperty:type:description:enum: takes the same arguments as the unconstrained variant plus an array of allowed values, and the model is constrained to emit one of them. Below, the status field is restricted to three states; the model cannot return "in transit" or any other off-vocabulary value:

```
"Enum-constrained string: status must be one of three fixed values."
```

```
| schema |
schema := ClaudeToolInputSchema new
    addProperty: 'orderId' type: 'string' description: 'Order identifier';
    addProperty: 'status'
        type: 'string'
        description: 'Order fulfilment state'
        enum: #('pending' 'shipped' 'delivered');
    require: 'orderId';
    require: 'status';
    yourself.
```

The second is the *nested object shape*, used when a record contains a sub-record — a customer with an address, an invoice with a billing party, a shipment with origin and destination. The builder method `#addProperty:schema:` takes a property name and another `ClaudeToolInputSchema` and registers the sub-schema as the property's value. Each sub-schema is built the same way the top-level one is, so the pattern composes recursively to whatever depth the wire schema needs:

```
"Nested object: a Customer record with an embedded Address."
| address customer |
address := ClaudeToolInputSchema new
    addProperty: 'street' type: 'string' description: 'Street address';
    addProperty: 'city' type: 'string' description: 'City name';
    addProperty: 'postalCode' type: 'string' description: 'Postal code';
    require: 'street';
    require: 'city';
    yourself.
customer := ClaudeToolInputSchema new
    addProperty: 'name' type: 'string' description: 'Customer name';
    addProperty: 'address' schema: address;
    require: 'name';
    require: 'address';
    yourself.
```

The third is the *array-of-record payload*, used when the response is a homogeneous list of records — order line items, a slice of search results, a batch of extracted entities. Set the schema's top-level `type` to `'array'` via `#type:`, then attach the per-element schema with `#items:`. The resulting wire schema is `{type: "array", items: {...}}` and the model emits a JSON array whose entries each match the inner schema:

```
"Array of records: a list of order line items. Each item must
 carry sku and quantity; price is optional so back-orderable lines
 can be quoted later without failing schema validation."
| lineItem orderLines |
lineItem := ClaudeToolInputSchema new
    addProperty: 'sku' type: 'string' description: 'Stock keeping unit';
    addProperty: 'quantity' type: 'integer' description: 'Units ordered';
    addProperty: 'price' type: 'number' description: 'Unit price (optional for back-orderable
        lines)';
    require: 'sku';
    require: 'quantity';
    yourself.
orderLines := ClaudeToolInputSchema new
    type: 'array';
    items: lineItem;
    yourself.
```

The example illustrates the `#require:` versus optional distinction: a property registered with `#addProperty:type:description:` appears in the schema's `properties` map, but only properties named via `#require:` are listed in the wire schema's `required` array. The model is free to omit any property not in that array — here, `price` can be absent on a back-orderable line item without violating the schema.

For arbitrary JSON-Schema keywords the builder methods do not cover — `anyOf` and `oneOf` union types, format validators, length constraints, regex patterns — `#schemaField:put:` stores the keyword on the schema's `extraFields` dictionary so it round-trips through `#asJson`. The setter is selective: the four keys the schema already models as typed state — `type`, `description`, `properties`, `required` — are routed through their dedicated setters (`#type:`, `#description:`, `#properties:`, `#required:`) so internal state stays consistent; every other keyword lands in `extraFields` verbatim. This is `ClaudeToolInputSchema`'s escape hatch for the full JSON-Schema surface: anything the schema does not model as a typed builder method can be added by name. For example, `schema schemaField: 'oneOf' put: { stringSchema asJson. numberSchema asJson }` attaches a discriminated union as a top-level keyword, and the wire payload carries it through unchanged.

For where the request parameter fits on the Messages API payload see §4; for the shared `ClaudeToolInputSchema` used in tool definitions see §32.

Status: *Available*. Anthropic docs: https://docs.claude.com/en/docs/build-with-claude/structured-outputs.

30 Token Counting (Feature Guide)

Token counting before you send a request is a planning surface, not an API surface. This section is the feature-guide companion to §6 Count Tokens: §6 documents the `countTokens:` selector and its response shape, while this section covers the *why* — pre-flight cost quoting, UX guardrails that warn the user before a long turn overruns the model's input cap, context-window selection when deciding between a 200K-token model and a 1M-token model, and batch-job gating where skipping a request that would exceed a quota is cheaper than letting it fail at send time. The same count the billing path will meter is available before the request leaves the client, so the application can price, gate, and size the turn in one pre-flight call.

To estimate tokens in an application the caller builds the `ClaudeMessageRequest` the same way `sendMessage:` takes one, hands it to `countTokens:`, and reads `inputTokens` off the returned `ClaudeMessageTokensCount`. For the runnable call and the precise response shape see §6; the API surface is reference material and lives there rather than being duplicated here. The worksheet below uses the same call shape as the §6 example to drive a four-step cost-quoting flow.

The first sizing decision is which context window the request needs. Three families ship today: 200K is the universal baseline; 500K is the production-tier window on `ClaudeModel #opus47` and `#opus46`; 1M is the long-context window on Sonnet 4.6, Opus 4.6, and Opus 4.7 (which include it at standard pricing without a beta flag, per Anthropic's pricing page) and on Sonnet 4.5 (which gates it behind the `context-1m-2025-08-07` beta). The table below maps each window to the model families the SDK ships typed `ClaudeModel` accessors for, and the beta header (if any) the request must carry — callers reach the beta string through the typed catalog `ClaudeBetaHeader context1m`, which returns `'context-1m-2025-08-07'` (see §12). Each row lists the model families whose maximum input window reaches the row's size, and the beta column names the only gate the request must clear — the 1M opt-in for Sonnet 4.5. Larger-cap families appear in smaller-cap rows too because a 1M-capable model still serves any request that fits a smaller window; the SDK exposes no per-request knob to select a window below a model's maximum:

Window	Model families	Beta required
200K	Sonnet 4.5, Sonnet 4.6, Opus 4.x, Haiku 3.5, Haiku 4.5	none
500K	Opus 4.6, Opus 4.7	none
1M	Sonnet 4.6, Opus 4.6, Opus 4.7	none (standard pricing)
1M	Sonnet 4.5	`context-1m-2025-08-07`

The second decision is what each input token will cost. Anthropic publishes per-model rates per million tokens (MTok) on its pricing page; the figures below are base input and output rates retrieved from

https://docs.claude.com/en/docs/about-claude/pricing on 2026-04-22. Cache-write and cache-hit pricing is a separate axis covered in §22. Callers should re-check the console before quoting — Anthropic adjusts rates and the table below is a snapshot, not a contract:

Model	Input $/MTok	Output $/MTok
Haiku 3.5	$0.80	$4.00
Haiku 4.5	$1.00	$5.00
Sonnet 4.5	$3.00	$15.00
Sonnet 4.6	$3.00	$15.00
Opus 4.6	$5.00	$25.00
Opus 4.7	$5.00	$25.00

The pre-flight cost quote runs in four steps. First, build the `ClaudeMessageRequest` the way `#sendMessage:` takes one — same model, same conversation, same system prompt. Second, hand it to `#countTokens:` on the client; the call returns a `ClaudeMessageTokensCount` carrying the single `#inputTokens` field, the same value the billing path will meter when the request is sent. Third, multiply that count by the model's per-million input rate from the table above; output cost is bounded by `maxTokens`, so a worst-case ceiling multiplies it by the output rate. Fourth, surface the figure in the UI before the caller commits. The runnable below quotes a Sonnet 4.6 turn with 4096 max output tokens; the call shape mirrors the §6 runnable:

Pre-flight cost quote

```
| client request count inputRate outputRate inputCost ceiling |
client := ClaudeClient apiKey: 'sk-ant-...'.

"Step 1: build the request the same way sendMessage: takes one."
request := ClaudeMessageRequest new
    model: ClaudeModel sonnet46;
    maxTokens: 4096;
    addUserMessage: 'Summarise the attached transcript in three bullet points.';
    yourself.

"Step 2: count input tokens; same shape as the section 6 runnable."
count := client countTokens: request.
count inputTokens "=> Integer, e.g. 12384"

"Step 3: multiply by per-model rates from the published pricing table."
inputRate := 3.00. "Sonnet 4.6 input: $3.00 / MTok (retrieved 2026-04-22)"
outputRate := 15.00. "Sonnet 4.6 output: $15.00 / MTok"
inputCost := count inputTokens * inputRate / 1000000.
ceiling := inputCost + (request maxTokens * outputRate / 1000000).

"Step 4: surface the quote in the UI."
String streamContents: [ :s |
    s nextPutAll: 'Input cost: $';
      nextPutAll: (inputCost printShowingDecimalPlaces: 4);
      nextPutAll: ' (worst case with maxTokens output: $';
      nextPutAll: (ceiling printShowingDecimalPlaces: 4);
      nextPutAll: ')' ]
```

Two tactics extend the same flow. Batch jobs gate at step 2: if `#inputTokens` exceeds the per-job quota the orchestrator skips the send entirely, paying nothing instead of a 400 round-trip. UX guardrails fire between steps 2 and 4: if the count is within 90% of the chosen model's context window, the application warns the user before committing rather than after the model truncates. Both tactics use the same `#countTokens:` call — the planning surface is one endpoint, the policy is application-side.

For the `countTokens:` API reference and the runnable example see §6; for the beta-header mechanism the 1M context-window beta uses see §11; for the prompt-caching pricing axis see §22.

Status: *Available*. Anthropic docs: https://docs.claude.com/en/docs/build-with-claude/token-counting.

31 Batch Processing

Batch processing is the feature-guide companion to §16 Batches API: §16 catalogues the six operations (create, retrieve, retrieve results, list, cancel, delete) that would land once the surface ships, while this section covers the *why* — overnight evaluation runs against large question banks, bulk classification jobs over warehouse exports, model-versus-model A/B comparisons over fixed test sets, and retrospective backfills where days of historical traffic replay through a new prompt. Applications reach for the Batches API when the workload tolerates up to 24 hours of completion latency in exchange for the discounted batch service tier, which prices per-token consumption below the interactive `auto` and `standard_only` tiers. The batch tier is itself not a `service_tier` request-field value (see §14); it is intrinsic to the batch endpoint and applies to every request inside the job automatically.

The Pharo SDK does not implement the Batches API today. See §16 for the planned surface and the implementation shape the Pharo layer would take when the feature lands — a typed request/response layer in `Claude-Messaging-Types`, `ClaudeClient` selectors for the six operations, and a streaming reader for the results download so jobs with 100 000 per-request results do not materialize in full in VM memory. §3 lists the Batches API among the v2 surfaces deferred beyond the v1 release.

This section is reference material; no runnable example applies.

For the API inventory and the planned implementation shape see §16; for the distinction between the batch endpoint and the per-request `service_tier` field see §14.

Status: *Not yet implemented*. Anthropic docs: https://docs.claude.com/en/docs/build-with-claude/batch-processing.

Part IV

Agents and Tools

32 Tool Use

Tool use turns the Messages API into a two-way conversation between Claude and application code. The application declares one or more tools on the request — each one a name, a prose description, and a JSON Schema for its inputs. When Claude decides it needs the tool, it returns a `ClaudeToolUseBlock` in the response content; the caller runs whatever the tool does, wraps the answer in a `ClaudeToolResultBlock`, appends it to the message history, and sends again. Claude integrates the result and continues. The loop is what powers agentic applications: database lookups, API calls, file reads, shell commands, anything the model can't do on its own.

The messaging layer ships the primitives for building one turn of that loop. A tool is a `ClaudeTool` — three slots (name, description, input schema), plus an optional `cacheControl` marker for caching tool definitions across requests (§22). The input schema is a `ClaudeToolInputSchema`, a builder over the JSON Schema shape the Messages API expects: type defaults to `'object'`, a `properties` dictionary holds the parameters, `required` lists the mandatory keys. A tool is attached to a request via `ClaudeMessageRequest>>tools:`, which takes a collection. When Claude uses the tool, the matching content block in the response answers `isToolUseBlock` with `true` and exposes `name` (the tool it picked) and `input` (the arguments it supplies as a Dictionary). The next turn replies with a `ClaudeToolResultBlock` whose `toolUseId` links back to the block Claude emitted.

The example below shows one turn end-to-end: declare a `get_weather` tool with a single `city` string property, attach it to a request, send, and dispatch on the response content. The full round-trip loop — detect the `tool_use` block, run the tool locally, append a `tool_result` block, send again, repeat until Claude stops requesting tools — lives in the Claude Agent SDK via `ClaudeExchange`. This section shows only the messaging primitives; `ClaudeExchange` is specified in the *Claude Agent SDK Specification*.

```
| client schema tool request response |
client := ClaudeClient apiKey: 'sk-ant-...'.

"Build a JSON Schema with one string property."
schema := ClaudeToolInputSchema new
    addProperty: 'city' type: 'string' description: 'City name';
    yourself.

"Wrap name, description, and schema in a ClaudeTool."
tool := ClaudeTool
    name: 'get_weather'
    description: 'Get current weather for a city'
    inputSchema: schema.

"Attach the tool to a request via tools: (takes a collection)."
request := ClaudeMessageRequest new
    model: ClaudeModel sonnet46;
    maxTokens: 1024;
    tools: (Array with: tool);
    addUserMessage: 'What is the weather in Paris?';
    yourself.

"Send. Claude may answer with a ClaudeToolUseBlock in the response content."
response := client sendMessage: request.
response content do: [ :block |
    block isToolUseBlock ifTrue: [
        "block name => 'get_weather' "
        "block input => a Dictionary('city'->'Paris' ) "
        Transcript show: 'Call ', block name, ' with ', block input printString; cr ] ]
```

The snippet above was run against a live Pharo image. Constructing `tool` yields `'get_weather'` as its `name`; attaching it via `tools:` produces a serialized request whose `tools` array has one entry; a synthesized `ClaudeToolUseBlock` answers `isToolUseBlock` with `true` and round-trips its `name` and `input` slots, with `input` printing as a Pharo Dictionary (`#(true 'get_weather' 'a Dictionary(''city''->''Paris'' )')`).

Phase 2 adds the tool-choice modes (`any`, `required`, `tool`), `disableParallelToolUse`, fine-grained tool-input streaming, and the token-efficient-tools beta identifier. The messaging primitives above do not change; the request-shaping setters land on `ClaudeMessageRequest` and `ClaudeRequestOptions`.

For the Anthropic-hosted tool siblings see §33; for external tools bridged through Anthropic see §34; for the request/response shape the tools ride on see §4; for thinking blocks that can interleave with tool-use turns see §23; the full tool-use loop is specified in the *Claude Agent SDK Specification* via `ClaudeExchange`.

Status: *Available*. Anthropic docs: https://docs.claude.com/en/docs/agents-and-tools/tool-use/overview. Python SDK peer: https://docs.anthropic.com/en/docs/build-with-claude/tool-use/overview.

33 Server Tools

Server tools flip the execution boundary. Instead of Claude asking the application to run a tool and waiting for the answer, Anthropic hosts the tool and runs it directly during message generation. The application declares which hosted tools are allowed on the request; the API invokes them as needed and returns the results inline with the rest of the response content. No round-trip through application code is needed for web search, web fetch, sandboxed code execution, sandboxed bash, a server-side text editor, persistent memory, or BM25/regex tool search. Server tools are what you reach for when you want Claude to *do* something the API can perform on your behalf, rather than delegate the work back to your stack.

The Smalltalk SDK models each hosted tool as a concrete subclass of `ClaudeServerTool` in `Claude-Messaging-Tools`. Each class carries the wire-format name, defaults for any tool-specific parameters, and an `asJson` method that serializes the tool declaration Anthropic expects in the request's `tools` array. The hierarchy is small and flat:

Class	Tool Name	Purpose
`ClaudeServerTool`	(abstract)	Base class
`ClaudeWebSearchTool`	`web_search`	Web search
`ClaudeWebFetchTool`	`web_fetch`	Fetch URL content
`ClaudeCodeExecutionTool`	`code_execution`	Sandboxed code execution
`ClaudeBashTool`	`bash`	Sandboxed bash
`ClaudeTextEditorTool`	`text_editor`	Server-side text editor
`ClaudeMemoryTool`	`memory`	Persistent memory
`ClaudeToolSearchBm25Tool`	`tool_search_bm25`	BM25 tool search
`ClaudeToolSearchRegexTool`	`tool_search_regex`	Regex tool search

`ClaudeMessageRequest` gains an extension method per tool in `Claude-Messaging-Tools` — `addWebSearch`, `addWebFetch`, `addCodeExecution`, `addBash`, `addTextEditor`, `addMemory`, `addToolSearchBm25`, `addToolSearchRegex` — each one a one-line wrapper that instantiates the matching subclass with defaults and appends it to the request's tool list. Because the extension lives in the tools package, the core `Claude-Messaging-Types` package stays free of dependencies on the server-tool hierarchy; loading `Claude-Messaging-Tools` into the image is what lights the convenience methods up. Server tools can coexist with client-defined tools (§32) on the same request: both land in the same `tools` array, and Anthropic dispatches to the right executor by tool name.

```
| client params response |
client := ClaudeClient apiKey: 'sk-ant-...'.

params := ClaudeMessageRequest new
    model: ClaudeModel sonnet46;
    maxTokens: 1024;
    addUserMessage: 'Summarise today''s headline at anthropic.com';
    yourself.

"Four one-liners, each appending a ClaudeServerTool subclass to the tools array."
params addWebSearch.
params addWebFetch.
params addCodeExecution.
params addBash.

"The request now carries four hosted-tool declarations."
(params asJson at: 'tools') size. "=> 4"

"Send. Anthropic runs the tools server-side and returns their results inline."
response := client sendMessage: params.
```

Reference docs for each hosted tool live under https://docs.claude.com/en/docs/agents-and-tools/tool-use/ (`web-search-tool`, `web-fetch-tool`, `code-execution-tool`, `bash-tool`, `text-editor-tool`, plus Memory and Tool Search sub-pages). Tool-specific betas — when Anthropic gates a new server tool behind an opt-in — ride on `betas:` in the request the same way every other beta does (§12).

Phase 2 evaluates Computer Use (`computer_20250124`) and Anthropic's programmatic tool-calling shape; neither ships in the current release.

For the client-defined tool sibling see §32; for external tools bridged through an MCP server see §34; for the opt-in header mechanism see §12.

Status: *Available*. Anthropic docs: https://docs.claude.com/en/docs/agents-and-tools/tool-use/. Python SDK peer: https://docs.anthropic.com/en/docs/build-with-claude/tool-use/overview.

34 MCP Connector

The MCP connector beta lets the SDK point Claude at a Model Context Protocol server hosted somewhere on the public internet, and have Anthropic's infrastructure — not the application's runtime — connect out to that server during message generation. The caller lists one or more server URLs on the request under `mcpServers:`, opts into the `mcp-client-2025-04-04` beta via `betas:`, and sends. Anthropic plays the role of MCP client; when the model invokes a tool on one of the listed servers, Anthropic dispatches the call, collects the result, and folds both the invocation and the result into the response stream as new content-block types the SDK dispatches on. The MCP server itself runs externally, on the customer's infrastructure or a third-party host.

Three features share the MCP acronym in this codebase, and only one of them is this connector. *This* connector is the SDK-side feature where Claude consumes MCP tools through Anthropic; exposing the Pharo image *as* an MCP server for another MCP host to call is a separate, post-v1.0 feature; the local MCP runtime Claude Code uses to back slash commands like `/find` or `/who` is a different surface entirely (biff, quarry, lux plugins, not an SDK concern).

The Smalltalk surface lives in two packages. Request-side class definitions are in `Claude-Messaging-MCP`; the response-side content blocks live in `Claude-Messaging-Types` alongside the other `ClaudeContentBlock` subclasses, registered in the name-keyed `TypeRegistry` under `mcp_tool_use` and `mcp_tool_result`:

Class	Role
`ClaudeMCPServerDefinition`	Request-side server entry: name, URL, optional auth token, optional tool config
`ClaudeMCPToolConfiguration`	Per-server tool config (enabled flag, allowlisted tool names)
`ClaudeMCPToolUseBlock`	Response block when Claude invokes a tool on the MCP server
`ClaudeMCPToolResultBlock`	Response block carrying the result (String or `OrderedCollection` of `ClaudeTextBlock`)

The `ClaudeMCPToolResultBlock content` slot is deliberately polymorphic: a wire `string` stays a `String`; a wire array becomes an `OrderedCollection` of `ClaudeTextBlock`. `asJson` dispatches on the runtime shape so a `fromJson:/asJson` round-trip preserves wire fidelity in both directions. Both `ClaudeMCPServerDefinition` and `ClaudeMCPToolConfiguration` carry an `extraFields` slot for forward-compat preservation of unknown wire keys, mirroring the strategy `ClaudeContentBlock` uses for unrecognized block types.

The SDK does not auto-inject the MCP beta header when `mcpServers:` is set; the caller opts in explicitly via `betas: #('mcp-client-2025-04-04')`. Explicit opt-in matches the shape of the Files API, PDF document blocks, and the `output-128k` beta — the SDK never activates a beta on the caller's behalf. Two required-field guards fail fast before the request leaves the client: `asJson` on `ClaudeMCPServerDefinition` raises an `Error` (via `self error:`) when `name` or `url` is `nil`, and on `ClaudeMCPToolResultBlock` when `toolUseId` is `nil`. Both are required on the wire; failing in-image is preferable to a `400` round-trip.

```
| client server params response |
client := ClaudeClient apiKey: 'sk-ant-...'.

"Build a server entry with name, URL, and optional bearer token."
server := ClaudeMCPServerDefinition new
    name: 'example';
    url: 'https://mcp.example.com/sse';
    authorizationToken: 'token-123';
    yourself.

"Opt into the MCP beta and attach the server list on the request."
```

```
params := ClaudeMessageRequest new
    model: ClaudeModel sonnet46;
    maxTokens: 1024;
    betas: #('mcp-client-2025-04-04');
    mcpServers: (Array with: server);
    addUserMessage: 'List my open issues';
    yourself.

"Send. Anthropic connects out to the MCP server when the model needs it."
response := client sendMessage: params.

"The response content may carry new block types; dispatch on the predicates."
response content do: [ :block |
    block isMCPToolUseBlock ifTrue: [
        "block serverName, block name, block input carry the invocation" ].
    block isMCPToolResultBlock ifTrue: [
        "block toolUseId links back; block content is String or
         OrderedCollection of ClaudeTextBlock" ] ]
```

For the client-defined tool sibling see §32; for Anthropic-hosted tools where the API executes the tool directly see §33; for the opt-in header mechanism see §12.

Status: *Available*. Anthropic docs: https://docs.claude.com/en/docs/agents-and-tools/mcp-connector.

35 Agent Skills

An Agent Skill is a portable bundle of instructions and supporting scripts that Anthropic loads into a code-execution container on Claude's behalf during a Messages request. Each skill is a directory rooted at a `SKILL.md`, uploaded as a versioned artifact through the Skills API; a request that references the skill id inside its container spec gets the files unpacked into the container's filesystem and made visible to the model. Anthropic ships a pre-built catalog — `pptx`, `xlsx`, `docx`, `pdf` — for document generation, and customers can publish their own skills for any repeatable capability they want Claude to reach for on the next turn. The pre-built catalog is server-side: the SDK does not enumerate the pre-built names locally, it just loads any skill by id into the request's container.

Skills sit alongside §32 and §33 as a third kind of capability surface, and the choice between them comes down to what the capability *is*. A **tool** is a single-purpose callable the model invokes mid-turn through the tool-use protocol — the model emits a `tool_use` block naming the tool and its arguments, the caller executes the tool, and the model receives a `tool_result` on the next turn. A **skill** is a versioned bundle of prose and supporting scripts that Anthropic unpacks into the request's code-execution container; the model loads the bundle's `SKILL.md` and uses the included files however the task demands, with no per-step round-trip back to the caller. **Retrieval-augmented generation** (RAG) is a third neighbour that supplies reference material the model reads but does not execute — a similarity search over a corpus surfaces the documents most relevant to the current turn, and those documents land in the context window as plain text. Tools execute, skills are loaded and used, RAG retrieves and reads.

The decision between the three follows the shape of the request. "Query the weather API for Berlin" is a tool: a single call with typed arguments and a typed return value. "Generate a PowerPoint from these bullet points" is a skill — Anthropic's pre-built `pptx` catalog entry (see §18 for the catalog shape) bundles the templates and rendering scripts the model needs, and the call to the model carries no per-slide tool round-trips. "Find the three most similar past incidents in our postmortem archive" is RAG: the search runs against a vector index, the top-k documents enter the context, and the model reasons over them. "Compile a quarterly sales report by region" is again a skill — the `pptx` and `xlsx` catalog entries compose, the model loads both into one container, and the output is a workbook plus a slide deck the model assembles in one turn.

Skills, tools, and RAG compose freely. A skill's container can call tools the request also declares — the model can reach for the weather tool from inside a script the `pptx` skill loaded. A tool the caller authors

can synthesize skill output — the tool's implementation can post-process a workbook the model just built. And RAG complements both: the retrieved documents land in the prompt that drives the skill's container, or supply the context a tool's arguments encode. The three layers stack rather than compete.

This section is the feature-guide entry point for skills; the mechanics — upload, list, version, delete, container loading, page-token pagination, multipart streaming, skill specs in `ClaudeContainerParams` and `ClaudeSkillSpec` — are specified in §18. This section is reference material; no runnable example applies. A complete skill-upload example lives in §18.

For the Skills API mechanics see §18; for tools as an alternate capability shape see §32.

Status: *Available*. Anthropic docs: https://docs.claude.com/en/docs/agents-and-tools/agent-skills/overview.